

Este tipo de eventos implica una continua comprobación de la condición. Sin embargo, en la práctica el desarrollador puede realizar la comprobación en tiempos discretos bien definidos analizando aquellos momentos en que las entradas de la condición cambian, por lo que no es necesaria la comprobación continua.

El evento ocurre una vez cuando el valor de la condición cambia de falso a verdadero (cambio a positivo), y no vuelve a repetirse a menos que el primer valor vuelva a valer falso.

Obsérvese la diferencia con una condición de guarda. Una condición de guarda se evalúa una vez siempre que ocurre el evento de su transición. Si la condición es falsa, no se dispara la transición y el evento se pierde (a menos que ese evento dispare alguna otra transición). Un evento de cambio se evalúa continuamente de forma implícita y ocurre una vez cuando el valor toma el valor verdadero. En ese momento, puede disparar una transición o ser ignorado. Si es ignorado, el evento de cambio no dispara una transición en ningún estado siguiente simplemente porque la condición sea aún verdadera: el evento ya ocurrió y fue ignorado. La condición debe volver a tener el valor falso y pasar de nuevo a verdadero para volver a provocar un evento de cambio.

Los valores de la expresión booleana deben ser atributos del objeto propietario de la máquina de estados que contiene la transición o valores alcanzables desde él.

Notación

A diferencia de las señales, los eventos de cambio no tienen nombre y no se declaran. Simplemente se utilizan como disparadores de las transiciones. Un evento de cambio se representa mediante la palabra clave **when** seguida de una expresión booleana entre paréntesis. Por ejemplo:

```
when (self.clientesEsperando > 6)
```

Discusión

Un evento de cambio es una prueba de satisfacción de una condición. Puede ser costoso de implementar, aunque hay muchas técnicas que permiten compilarlo para que no se produzca una comprobación continua. No obstante, es potencialmente costoso, y además oculta la relación causa-efecto directa entre el cambio de valor y los efectos disparados por dicho cambio. Esto último a veces es deseable para así encapsular los efectos, pero los eventos de cambio deben utilizarse con cuidado.

Un evento de cambio está pensado para representar un chequeo de valores visible para un objeto. Si un cambio en un atributo de un objeto dispara un cambio en otro objeto que no es consciente siquiera de la existencia del atributo, la situación debería modelarse como un evento de cambio sobre el propietario del atributo, que dispara una transición interna, la cual a su vez envía una señal al segundo objeto.

Obsérvese que un evento de cambio no se envía explícitamente a ningún sitio. Si se pretende modelar una comunicación explícita con otro objeto, debería utilizarse en su lugar una señal.

La implementación de un evento de cambio se puede llevar a cabo de varias formas, alguna de ellas haciendo pruebas dentro de la propia aplicación para calcular los tiempos de chequeo más adecuados, y otras utilizando facilidades del sistema operativo subyacente.

evento de llamada

Evento por el que se recibe una llamada para realizar una operación. Se implementa mediante acciones o transiciones de la máquina de estados.

Véase también llamada, señal.

Semántica

Un evento de llamada es una forma de implementar una operación, distinta de la ejecución de un procedimiento. Si una clase especifica la implementación de una operación como un evento de llamada, toda llamada a la operación será tratada como un evento que dispara una transición en la máquina de estados de la clase. Esto permite una implementación más dispersa, en contraposición al método monolítico de los procedimientos, que siempre hacen lo mismo (un procedimiento puede tener una sentencia de selección múltiple, por lo que no hay una diferencia real en potencia entre los dos enfoques).

Si una clase utiliza eventos para implementar una operación, su máquina de estados debe tener transiciones disparadas por el evento de llamada. La signatura de un evento de llamada es la misma que la de una operación: el nombre del evento de llamada sería el nombre de la operación, mientras que los parámetros del evento de llamada serían los parámetros de la operación.

Cuando se produce un evento de llamada, se consulta la máquina de estados del objeto destino y si el evento de llamada se corresponde con un evento disparador en una transición activa (uno que salga de un estado activo actual) se dispara una transición. Si se dispara una transición, se ejecutan sus efectos, que pueden incluir cualquier secuencia de acciones, incluyendo una acción **return**(valor), cuyo propósito es devolver un valor a quien realizó la llamada.

Cuando finaliza la ejecución de la transición, quien realizó la llamada retoma el control y puede continuar la ejecución. Si la operación requiere devolver un valor de retorno y el que realizó la llamada no lo recibe, o recibe un valor inconsistente con el tipo declarado en la operación, el modelo es erróneo. Si la operación no requiere valor de retorno no hay problema. Obsérvese que si un evento de llamada no dispara ninguna transición, el control retorna inmediatamente a quien hizo la llamada; si la operación requiere devolver un valor, el modelo es erróneo; y si no, el que realiza la llamada se reanuda inmediatamente.

Si el receptor es un objeto activo, un evento de llamada se trata cuando la máquina de estados del receptor esté inactiva, es decir, cuando hayan finalizado todos los pasos de la ejecución a completar.

Notación

Un evento de llamada se representa mediante un evento disparador en un diagrama de estados que se corresponde con una operación de la clase correspondiente. La secuencia de acciones de la transición puede tener una sentencia **return**; si es así, la sentencia debe indicar el valor devuelto.

texto del procedimiento de llamada

ingresar (10);

...

cantidad := disponer ()

Sacar todo o nada
dependiendo de si la cuenta
está bloqueada o no

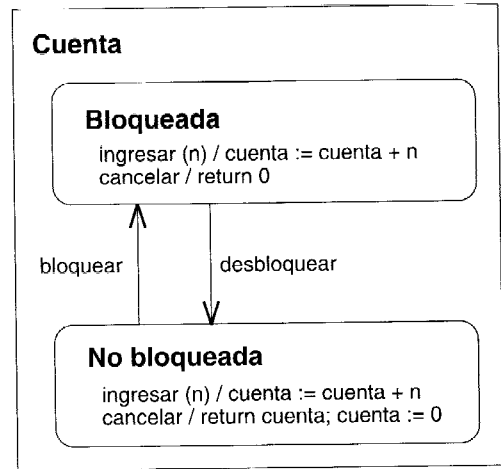


Figura 13.103 Eventos de llamada

Ejemplo

La Figura 13.103 muestra una cuenta que puede estar **Bloqueada** y **NoBloqueada**. La operación **ingresar** siempre añade dinero a la cuenta, sea cual sea su estado. La operación **disponer**, permite sacar todo el dinero de la cuenta si no está bloqueada, pero si lo está no es posible sacarlo. La operación **disponer** está implementada como un evento de llamada que dispara transiciones internas en cada uno de los estados. Cuando se produce la llamada, se ejecuta una u otra secuencia de acciones, dependiendo del estado activo. Si el sistema está bloqueado, se devuelve cero; si no lo está, se devuelve todo el dinero que tenía la cuenta y la cuenta se pone a cero.

evento de señal

Un evento que es la recepción por parte de un objeto de una señal que se le envía y que puede disparar una transición en su máquina de estados.

evento diferido

Un evento cuyo reconocimiento se retrasa mientras un objeto está en cierto estado.

Véase también máquina de estados, transición.

Semántica

Un estado puede señalar un conjunto de eventos como diferidos. Si ocurre un evento mientras un objeto está en un estado que ha diferido el evento y el evento no acciona una transición, el evento no tiene ningún efecto inmediato. Se almacena hasta que el objeto entre en un estado en el cual el evento en cuestión no esté diferido. Si ocurren otros eventos mientras el estado está

activo, se manejan de manera general. Cuando el objeto entra en un nuevo estado, los eventos guardados que ya no son diferidos ocurren uno tras otro y pueden accionar transiciones en el nuevo estado (el orden de ocurrencia de los eventos previamente diferidos es indeterminado, es aventurado depender de un orden de ocurrencia en particular). Si el evento no acciona ninguna transición en el estado no diferido, entonces es ignorado y se pierde.

Los eventos diferidos se deben utilizar con cuidado en máquinas de estados ordinarias. Pueden ser modelados a menudo más directamente por un estado concurrente que responda a ellos mientras que el elemento computacional principal está haciendo algo más. Pueden ser útiles en los estados de actividad en los cuales permiten que los cálculos se realicen secuencialmente sin perder mensajes asincrónicos.

Si un estado tiene una transición accionada por un evento diferido, entonces la transición elimina el aplazamiento y el evento acciona la transición.

Notación

El evento diferido se indica por una transición interna en el evento con la acción reservada especial **defer** (diferir). El aplazamiento se aplica al estado y a sus subestados anidados (Figura 13.104).

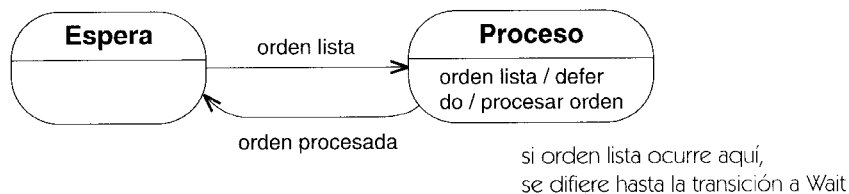


Figura 13.104 Evento diferido

evento temporal

Evento que denota el cumplimiento de alguna expresión temporal, tal como la aparición de un tiempo absoluto o el paso de un determinado período de tiempo una vez que un objeto entra en un estado.

Semántica

Un evento temporal es un evento que depende del paso del tiempo y por tanto de la existencia de un reloj. En el mundo real, el reloj es algo implícito. En un computador, se trata de una entidad física y puede haber distintos relojes en distintos computadores. El evento temporal es un mensaje del reloj al sistema. Obsérvese que se puede definir tanto el tiempo absoluto como el relativo con respecto a un reloj del mundo real o bien con respecto a un reloj interno virtual (en este último caso, puede ser distinto para diferentes objetos).

Los eventos temporales pueden estar basados en un tiempo absoluto (la hora del día o algún ajuste del reloj dentro de un sistema) o bien en tiempos relativos (el tiempo transcurrido desde la entrada en un cierto estado o bien desde que se ha producido un evento).

Notación

Los eventos temporales no se declaran como eventos con nombre tal como se hace con las señales. En lugar de hacer esto, se emplea simplemente una expresión temporal como disparador de la transición.

Discusión

En cualquier implementación real, los eventos temporales no provienen del universo; vienen de algún objeto reloj situado dentro o fuera del sistema. Como tales, son casi indistinguibles de señales, especialmente en los sistemas distribuidos y de tiempo real. En tales sistemas también hay que resolver cuál de los relojes se va a utilizar —no hay nada que sea el “tiempo real”—. (Tampoco existe en el universo real —pregúntele a Einstein—.)

excepción

Una señal levantada en respuesta a fallos del comportamiento de la ejecución de la máquina subyacente.

Véase también estado compuesto, señal.

Semántica

Una excepción se genera generalmente implícitamente, por los mecanismos subyacentes de la implementación, en respuesta a una falta durante la ejecución. Puede considerarse como una señal al objeto activo o procedimiento de activación que causó la ejecución. La máquina de estados del objeto puede tomar una acción apropiada para gestionar la excepción, incluyendo abortar el proceso actual e ir a un lugar conocido en la ejecución, ejecutar una operación sin un cambio de estado, o ignorar el evento. La capacidad de unir transiciones a los estados de alto nivel hace la gestión de excepciones flexible y potente.

Una excepción es una señal y por lo tanto tiene una lista de los parámetros que están limitados a valores cuando ocurre la excepción. Los valores de parámetros son fijados por la maquinaria de la ejecución que detecta la avería, tal como el sistema operativo. La operación que maneja la excepción puede leer los parámetros. En la mayoría de los lenguajes, las excepciones pueden ser manipuladas y retransmitidas por las operaciones que las manejan.

Notación

El estereotipo «**exception**» puede utilizarse para distinguir la declaración de una excepción. No es necesario un estereotipo para usar el nombre del evento en una máquina de estados.

exportar

En el contexto de los paquetes, hacer un elemento accesible fuera del espacio de nombres que lo contiene ajustando su visibilidad. Contrasta con el acceso y la importación, los cuales hacen a elementos exteriores accesibles dentro de un paquete.

Véase también acceder, importar, visibilidad.

Semántica

Un paquete exporta un elemento fijando su visibilidad a un nivel que permita que sea visto por otros paquetes (público para los paquetes que lo importan, protegido para sus propios hijos).

Discusión

Son necesarias dos cosas para que un elemento (tal como una clase) pueda ver un elemento en un paquete del mismo nivel. El paquete que contiene el elemento objetivo debe exportarlo dándole visibilidad pública. Además, el paquete que se refiere al elemento objetivo debe acceder o importar el paquete que contiene el elemento objetivo. Ambos pasos son necesarios.

expresión

Una cadena que codifica una sentencia que se interpretará por un lenguaje dado. Muchos tipos de expresiones proporcionan valores cuando son interpretadas; otros tipos realizan acciones específicas. Por ejemplo, la expresión “ $(7 + 5 * 3)$ ” se evalúa a un valor de tipo número.

Semántica

Una expresión define una sentencia que evalúa un conjunto (posiblemente vacío) de instancias o de valores o el resultado de una acción específica cuando se ejecuta en un contexto. Una expresión no modifica el entorno en el cual se evalúa. Una expresión consiste en una cadena y el nombre del lenguaje de la evaluación.

Un elemento de la expresión contiene el nombre del lenguaje de interpretación y de la cadena que codifica la expresión en la sintaxis del lenguaje elegido. Se asume que un intérprete está disponible para el lenguaje. Proporcionar un intérprete es responsabilidad de la herramienta que modela. El lenguaje puede ser un lenguaje de especificación de restricciones, como OCL; puede ser un lenguaje de programación, como C++ o Smalltalk; o puede ser un lenguaje natural. Por supuesto, si se escribe una expresión en lenguaje natural, no puede ser evaluado automáticamente por una herramienta y deberá ser totalmente para consumo humano.

Las diferentes subclases de expresiones producen diversos tipos de valores. Éstos incluyen expresiones booleanas, expresiones de conjuntos de objetos, expresiones de tiempo, y expresiones de procedimiento.

Las expresiones aparecen en modelos de UML como acciones, restricciones, condiciones de guarda, y otros.

Notación

Se muestra una expresión como una cadena definida en un lenguaje. La sintaxis de la cadena es responsabilidad de una herramienta y de un analizador lingüístico para el lenguaje. La asunción es que el analizador puede evaluar cadenas en tiempo de ejecución para producir valores del tipo apropiado, o puede proporcionar las estructuras semánticas para capturar el significado de la expresión. Por ejemplo, una expresión de tipo evalúa una referencia del clasificador, y una expresión booleana evalúa un valor verdadero o falso. Se conoce el lenguaje para una herramienta de modelado, pero generalmente es implícito en el diagrama bajo la asunción de que la forma de la expresión hace claros sus propósitos.

Ejemplo

```
self.coste < autorización.costeMáximo
para todo (k en destinos) { k.actualizar () }
```

expresión booleana

Expresión que se evalúa a un valor booleano. Útil en condiciones de guarda.

expresión de acción

Expresión que puede estar compuesta por una acción o por una secuencia de acciones.

Discusión

UML no especifica la sintaxis de una expresión de acción, pues eso es responsabilidad de las herramientas que trabajen con UML. Se espera con ello que los diferentes usuarios puedan expresar acciones mediante lenguajes de programación, pseudocódigo o incluso mediante lenguaje natural. Para el diseño detallado se necesita una sintaxis más precisa, por supuesto, pero es ahí donde la mayoría de los usuarios utilizarán los lenguajes de programación reales.

expresión de actividad

Expresión textual de un cómputo no atómico (una actividad). Una expresión de este tipo se puede descomponer conceptualmente en fragmentos atómicos, pero es conveniente permitir una representación textual de la actividad completa. La ejecución de una expresión de actividad puede ser finalizada por una transición que desactive el estado de control.

Semántica

Una expresión de actividad es un procedimiento efectivo (algoritmo) expresado en algún lenguaje, que puede ser un lenguaje de programación u otro tipo de lenguaje formal, aunque también puede expresarse mediante lenguaje natural. Si se utiliza el lenguaje natural, no será posible ejecutarlo con herramientas ni comprobar sus errores u otras propiedades, pero puede ser suficiente en las primeras etapas de trabajo. También puede representar una operación continua del mundo real.

Notación

Una expresión de actividad se representa como un texto interpretado en algún lenguaje.

Ejemplo

do / invertirMatriz	Finito pero tarda un tiempo
do / calcularMejorMovimiento	Se ejecuta hasta que termina el tiempo
do / sonar sirena	Continuo hasta que se para

expresión de conjunto de objetos

Indica una expresión que produce un conjunto de objetos cuando se evalúa en tiempo de ejecución.

Semántica

El blanco de una acción de envío es una expresión de conjunto de objetos. Cuando se evalúa una de estas expresiones en tiempo de ejecución, produce un conjunto de objetos al cual se envía en paralelo una señal acordada. Una expresión de conjunto de objetos puede proporcionar un único elemento, en cuyo caso la acción de envío es una acción secuencial ordinaria. Puede, incluso, no producir elemento alguno (esto es, un conjunto vacío) en cuyo caso no se produce el envío.

expresión de iteración

Una expresión que proporciona un conjunto de casos de iteración. Cada caso de iteración especifica la ejecución de una acción dentro de una iteración. Un caso de iteración puede incluir la asignación de valores a una variable de iteración. La acción se ejecuta una sola vez para cada caso de iteración.

Véase también mensaje.

Semántica

La expresión de iteración representa una ejecución condicional o iterativa. Representa la ejecución de cero o más mensajes, dependiendo de las condiciones implicadas. Las opciones son:

* [cláusula de iteración]	Una iteración
[cláusula de condición]	Una bifurcación

Una iteración representa una secuencia de mensajes. La cláusula de iteración muestra los detalles de la variable y de la prueba de iteración, pero se puede omitir (en cuyo caso las condiciones de iteración quedan sin especificar). La cláusula de iteración debería expresarse en pseudocódigo o en algún lenguaje real de programación. UML no prescribe su formato. Un ejemplo podría ser:

* [i:= 1..n]

Las condiciones representan un mensaje cuya ejecución es contingente respecto a la veracidad de la cláusula de condición. La cláusula de condición debería expresarse en pseudocódigo o en algún lenguaje real de programación. UML no prescribe su formato. Un ejemplo podría ser:

[x > y]

Obsérvese que las bifurcaciones se denotan igual que las iteraciones sin un asterisco. Se puede pensar que se trata de una iteración limitada a un único caso.

La notación de iteración supone que los mensajes de la iteración serán ejecutados secuencialmente. También existe la posibilidad de ejecutarlos concurrentemente. La notación es un asterisco seguido por una doble línea vertical, para denotar paralelismo (*||). Por ejemplo:

* [i:= 1..n]|| q[i].CalcularResultados ()

Obsérvese que en una estructura de control anidada la expresión de iteración no se repite en los niveles interiores del número de secuencia. Cada nivel de estructura especifica su propia iteración dentro del contexto que lo encierra.

expresión de procedimiento

Dícese de una expresión cuya evaluación representa la ejecución de un procedimiento que puede afectar al estado del sistema en ejecución.

Semántica

Una expresión de procedimiento es una codificación de un algoritmo ejecutable. Su ejecución puede (y suele) afectar al estado del sistema, esto es, tiene efectos secundarios. En general, las expresiones de procedimiento no proporcionan valores. El propósito de su ejecución es alterar el estado del sistema.

expresión de tipo

Expresión que al evaluarse produce una referencia a uno o más tipos de datos. Por ejemplo, una declaración de tipo de atributo en un lenguaje típico de programación es una expresión de tipo.

Ejemplo

En C++, lo que sigue son expresiones de tipo:

Persona*

Pedido[24]

Boolean (*)(String,int)

expresión temporal

Expresión que al ser resuelta proporciona un valor temporal absoluto o relativo. Se utiliza para definir los eventos temporales.

Semántica

Casi todas las expresiones temporales son o bien un tiempo transcurrido desde la entrada en un estado o bien la aparición de un cierto instante absoluto. Las demás expresiones temporales deberán definirse de forma ad hoc.

Notación

Tiempo transcurrido. Un evento que denota el paso de una cierta cantidad de tiempo tras haber entrado en el estado que contiene la transición se denota mediante la palabra reservada **after** seguida por una expresión que se evalúa (durante el tiempo de modelado) para producir una cantidad de tiempo.

after (10 segundos)

after (10 segundos después de salir del estado A)

Si no se especifica un instante inicial, se entiende que es el tiempo transcurrido desde la entrada en el estado que contiene la transición.

Tiempo absoluto. Un evento que denota llegada de un instante absoluto se denota mediante la palabra reservada **when** seguida por una expresión booleana entre paréntesis que implica la hora.

when (fecha = 1 de enero de 2000)

extender

Una relación desde un caso de uso extensor a un caso de uso base, que especifica cómo se puede insertar el comportamiento definido para el caso de uso extensor en el comportamiento definido para el caso de uso base. El caso de uso extensor modifica incrementalmente el caso de uso base de forma modular.

Véase también punto de extensión, incluir, caso de uso, generalización de casos de uso.

Semántica

La relación de extensión conecta un caso de uso extensor a un caso de uso base. El caso de uso extensor, en esta relación, no es necesariamente un clasificador instanciable separado. Consiste en uno o más segmentos que describen las secuencias adicionales de comportamiento que modifican incrementalmente el comportamiento del caso de uso base. Cada segmento en un caso de uso extensor se puede insertar en una localización separada en el caso de uso base. La relación de extensión tiene una lista de los nombres de los puntos de extensión, igual en número al número de segmentos en el caso de uso extensor. Cada punto de extensión se debe definir en el caso de uso base. Cuando la ejecución de una instancia de un caso de uso alcanza una localización en el caso de uso base referenciada por el punto de extensión y se cumple cualquier condición en la extensión, entonces la ejecución de la instancia se puede transferir a la secuencia de comportamiento del segmento correspondiente del caso de uso extensor; cuando la ejecución del segmento de la extensión es completa, el control vuelve al caso de uso original en el punto referenciado.

Se pueden aplicar múltiples relaciones de extensión al mismo caso de uso base. Una instancia de un caso de uso puede ejecutar más de una extensión durante el curso de su vida. Si varios casos de uso extienden un caso de uso base en el mismo punto de extensión, entonces su orden relativo de ejecución no es determinista. Puede incluso haber múltiples relaciones de extensión entre los mismos casos de uso extensor y base, a condición de que la extensión se inserte en una localización diferente en la base. Las extensiones pueden incluso ampliar otras extensiones de una manera jerarquizada.

Un caso de uso extensor en una relación de extensión puede tener acceso y modificar los atributos definidos por el caso de uso base. El caso de uso base, sin embargo, no puede ver las extensiones y puede no tener acceso a sus atributos u operaciones. El caso de uso base define un marco modular en el cual puedan ser agregadas extensiones, pero la base no tiene visibilidad sobre las extensiones. Las extensiones modifican implícitamente el comportamiento del caso de uso base. Observe la diferencia con la generalización de casos de uso. Con la extensión, los efectos del caso de uso extensor se agregan a los efectos del caso de uso base en una instanciación del caso de uso base. Con la generalización, los efectos del caso de uso hijo se agregan a los efectos del caso de uso padre en una instanciación del caso de uso hijo, mientras que una instanciación del caso de uso padre no consigue los efectos del caso de uso hijo.

Un caso de uso extensor puede extender más de un caso de uso base, y un caso de uso base se puede ampliar con más de un caso de uso extensor. Esto no indica ninguna relación entre los casos de uso base.

Un caso de uso extensor puede ser él mismo la base en una relación de extensión, inclusión, o generalización.

Estructura (del caso de uso extensor)

Un caso de uso extensor contiene una lista de uno o más *segmentos de inserción*, cada uno de los cuales es una secuencia de comportamiento.

Estructura (del caso de uso base)

El caso de uso define un conjunto de puntos de extensión, cada uno de los cuales referencia una localización o un conjunto de localizaciones en el caso de uso base donde se puede insertar el comportamiento adicional.

Estructura (de la relación de extensión)

La relación de extensión tiene una lista de los nombres de los puntos de extensión, que deben estar presentes en el caso de uso base. El número de nombres debe igualar el número de segmentos en el caso de uso extensor.

La relación de extensión puede tener una condición, una expresión en términos de atributos del caso de uso base, o la ocurrencia de eventos tales como recibir de una señal. La condición se determina si el caso de uso extensor está realizado cuando la ejecución de una instancia del caso de uso alcanza una localización referida por el primer punto de extensión. Si la condición está ausente, entonces se considera siempre como verdadera. Si la condición para un caso de uso extensor está satisfecha, entonces la ejecución del caso de uso extensor procede. Si el punto de extensión se refiere a varias localizaciones en el caso de uso base, el caso de uso extensor se puede ejecutar en cualquiera de ellas.

La extensión se puede realizar más de una vez, si la condición sigue siendo verdadera. Todos los segmentos del caso de uso extendido se ejecutan el mismo número de veces. Si el número de ejecuciones debe ser restringido, se puede definir la condición acorde.

Semántica de ejecución

Cuando una instancia de un caso de uso que realiza un caso de uso base alcanza una localización en el caso de uso base, que es referenciado por una relación de extensión, entonces se evalúa la condición en la relación de extensión. Si es verdad o si está ausente, entonces se realiza el caso de uso extendido. En muchos casos, la condición incluye la ocurrencia de un evento o la disponibilidad de los valores necesarios para el propio caso de uso extendido, por ejemplo, una señal de un actor que comience el segmento de la extensión. La condición puede depender del estado de la instancia del caso de uso, incluyendo valores del atributo del caso de uso base. Si no ocurre el evento o la condición es falsa, la ejecución del caso de uso extendido no comienza. Cuando el funcionamiento de un segmento de la extensión se completa, la instancia del caso de uso continúa realizando el caso de uso base en el punto donde lo dejó.

Las inserciones adicionales del caso de uso extendido pueden ser realizadas inmediatamente si la condición se satisface. Si el punto de la extensión se refiere a localizaciones múltiples en el caso de uso base, la condición se puede satisfacer en cualesquiera de ellos. La condición puede llegar a ser verdad en cualquier localización dentro del conjunto.

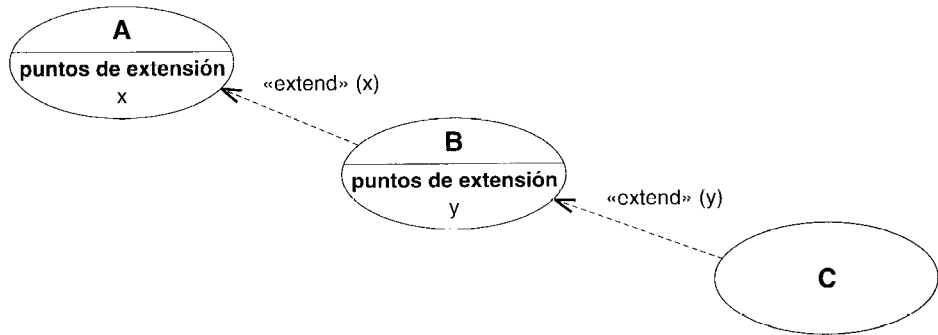


Figura 13.105 Extensiones anidadas

Si hay más de una cadena de una secuencia de inserción en un caso de uso extendido, entonces todos los segmentos de la inserción se ejecutan si la condición es verdad en el primer punto de la extensión. La condición no se vuelve a evaluar para los segmentos subsecuentes, los cuales se insertan cuando la instancia del caso de uso alcanza las localizaciones correspondientes dentro del caso de uso base. La instancia del caso de uso reasume la ejecución de la base entre las inserciones en diversos puntos de extensión. Una vez que estén comenzados, todos los segmentos deben ser realizados.

Observe que, en general, un caso de uso es una máquina de estados no determinista (como en una gramática), más que un procedimiento ejecutable. Eso es porque las condiciones pueden incluir la ocurrencia de eventos externos. Realizar un caso de uso como colaboración de clases puede requerir una transformación en mecanismos explícitos del control, así como la implementación de una gramática requiere una transformación a una forma ejecutable que sea eficiente pero más difícil de entender.

Observe que *base* y *extensión* son términos relativos. Una extensión puede servir por sí misma como base para otra extensión. Esto no presenta ninguna dificultad, y las reglas previas todavía se aplican —las inserciones se anidan—. Por ejemplo, suponga que un caso de uso B extiende un caso de uso A en el punto de extensión x, y suponga que el caso de uso C extiende

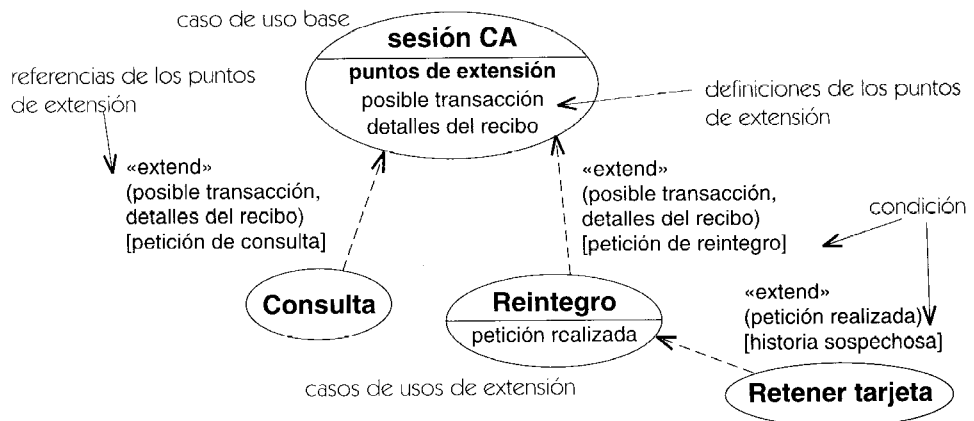


Figura 13.106 Relación de extensión

el caso de uso B en el punto de extensión y (Figura 13.105). Cuando una instancia de A llega al punto de extensión x, comienza a realizar el caso de uso B. Cuando la instancia entonces llega al punto de extensión y dentro de B, comienza a realizar el caso de uso C. Cuando se completa la ejecución de C, reasume la realización del caso de uso B. Cuando la ejecución de B está completa, reasume la ejecución de A. Es similar a llamadas anidadas a procedimientos o cualquier otra construcción anidada.

Notación

Se dibuja una flecha de línea discontinua desde el caso de uso extensor al símbolo del caso de uso base con una punta de flecha apuntando a la base. Se pone la palabra clave «**extend**» en la flecha.

Puede aparecer una lista de los nombres de los puntos de extensión entre paréntesis después de la palabra clave.

La Figura 13.106 muestra casos de uso con relaciones de extensión, y la Figura 13.107 muestra las secuencias de comportamiento de los casos de uso.

Caso de uso base para la sesión CA

mostrar el anuncio del día	
incluye (identificar cliente)	inclusión
incluye (validar cuenta)	inclusión
(el punto de extensión hace referencia aquí) <-----	<posible transacción>
imprimir cabecera del recibo	
(otro destino de un punto de extensión) <-----	<detalles del recibo>
desconexión	

Caso de uso base para la consulta

segmento	primer segmento
recibir petición de consulta	
mostrar información de la consulta	
segmento	segundo segmento
imprimir la información sobre el reintegro	

Caso de uso de extensión para la retirada de efectivo

segmento	primer segmento
recibir petición de retirada de efectivo	
especificar cantidad	
(otro punto de extensión destino) <-----	<petición realizada>
segmento	segundo segmento
desembolsar efectivo	

Caso de uso de extensión para tarjetas robadas

segmento	segmento único
engullir la tarjeta	
terminar la sesión	

Figura 13.107 Secuencias de comportamiento para los casos de uso

Discusión

Todas las relaciones de extensión, inclusión y generalización agregan comportamiento a un caso de uso inicial. Tienen muchas semejanzas, pero en la práctica es conveniente separarlas. La tabla 13.1 compara los tres puntos de vista.

Tabla 13.1 Comparación de las relaciones de casos de uso

<i>Propiedad</i>	<i>Extensión</i>	<i>Inclusión</i>	<i>Generalización</i>
comportamiento base	caso de uso base	caso de uso base	caso de uso padre
comportamiento añadido	caso de uso de extensión	caso de uso incluido	caso de uso hijo
dirección de referencia.	el caso de uso extensor referencia al caso de uso base	el caso de uso base referencia al caso de uso incluido	el caso de uso hijo referencia el caso de uso padre
¿base modificada?	la extensión modifica implícitamente el comportamiento de la base. La base debe estar formada sin extensión, pero si la extensión está presente, una instancia de la base puede ejecutar la extensión	la inclusión modifica explícitamente el efecto de la base. La base puede estar bien formada o no sin la inclusión, pero una instancia de la base ejecuta la inclusión	el efecto de ejecutar el padre no se modifica por el hijo. Para obtener efectos distintos, se debe instanciar el hijo y no el padre
¿instanciable?	la extensión no es necesariamente instanciable. Puede ser un fragmento	la inclusión no es necesariamente instanciable, puede ser un fragmento	el hijo no es necesariamente instanciable. Puede ser abstracto
¿puede acceder a los atributos de la base?	las extensiones pueden acceder y modificar el estado de la base	la inclusión puede acceder al estado de la base. La base debe proveer los atributos apropiados esperados por la inclusión	el hijo puede acceder y modificar el estado de la base (por los mecanismos usuales de herencia)
¿puede la base ver al otro?	la base no puede ver las extensiones y debe estar bien formada en su ausencia	la base ve la inclusión y puede depender de sus efectos, pero no puede acceder a sus atributos	el padre no puede ver al hijo, y debe estar bien formado en su ausencia
repetición	depende de la condición	exactamente una repetición	el hijo controla su propia ejecución

extensión

Conjunto de instancias descritas por un descriptor.

Contrastar con: intención.

Semántica

Un descriptor, tal como una clase o una asociación, tiene tanto una descripción (su intención) como un conjunto de los instancias que describe (su extensión). El propósito de la intención es especificar las características de las instancias que componen la extensión. No se asume que la extensión sea físicamente manifiesta o que pueda ser obtenida en tiempo de ejecución.

Por ejemplo, incluso en principio un modelador no debe asumir que el conjunto de los objetos que son instancias de una clase se puede obtener.

extremo de asociación

Parte estructural de una asociación que define la participación de una clase en la misma. Una clase puede estar conectada a más de un extremo en la misma asociación. Los extremos dentro de una asociación tienen distintas posiciones con diferentes nombres que, en general, no son intercambiables. Un extremo de asociación no tiene existencia independiente ni significado aparte de la asociación.

Véase también asociación.

Semántica

Estructura

Un extremo de asociación mantiene una referencia a un clasificador destino, y define la participación del clasificador en la asociación. Una instancia de la asociación (un enlace) debe contener, en una posición determinada, una instancia de la clase dada o de sus descendientes. Las clases hijas heredan la participación en una asociación.

Un extremo de asociación tiene las siguientes propiedades (véase cada entrada individual para más información):

agregación

Indica si el objeto asociado es un agregado o compuesto; es una enumeración con los valores {**none**, **aggregate**, **composite**}. Si el valor no es **none**, la asociación es una agregación o una composición. Éste es el valor por defecto. Sólo una asociación binaria puede ser agregación o composición, y sólo un extremo puede ser agregado o compuesto.

alcance destino	Indica si los enlaces relacionan objetos o clases enteras; es una enumeración con los valores { instance , classifier }. El valor por defecto es instance (relaciona objetos).
cualificador	Lista de atributos utilizada como selectores para encontrar objetos relacionados por una asociación.
especificador de interfaz	Restricción opcional sobre la especificación de tipo del objeto relacionado. Es un clasificador (hay quien llama a esto <i>rol</i> , aunque el término se utiliza en otros contextos).
multiplicidad	Indica el número posible de objetos que se pueden relacionar con un objeto, especificado normalmente como un rango de valores enteros.
navegabilidad	Valor lógico que indica si es posible atravesar una asociación binaria para obtener el objeto o conjunto de objetos asociados con una instancia de la clase. Por defecto es true (navegable).
nombre de rol	Nombre del extremo de asociación, que será un identificador de tipo cadena. Este nombre identifica el rol de la clase correspondiente dentro de la asociación. El nombre de rol debe ser único dentro de la asociación, y también entre pseudoatributos (atributos y otros nombres de rol visibles por la clase) directos y heredados de la clase origen.
ordenación	Expresa si (y potencialmente cómo) está ordenado el conjunto de objetos relacionados, es una enumeración con los valores { ordered , unordered }. También se puede utilizar el valor sorted con propósitos de diseño.
variabilidad	Indica si puede cambiar el conjunto de enlaces relacionados con un objeto mediante una lista con los valores { changeable , frozen , addOnly }. El valor por defecto es changeable .
visibilidad	Indica si el enlace puede acceder a otras clases además de aquellas que se encuentran en el extremo opuesto de la asociación. La visibilidad se sitúa en el extremo conectado a la clase destino. Cada dirección tiene su propio valor de visibilidad.

Notación

El extremo de la ruta de asociación se conecta con el borde del rectángulo correspondiente al símbolo de la clase. Las propiedades del extremo de asociación se representan como adornos sobre o cerca de la ruta en que se une a un símbolo de clasificador (véase Figura 13.108). La siguiente lista es un breve resumen de adornos para cada propiedad. Para más detalles, consultar individualmente cada uno de ellos.

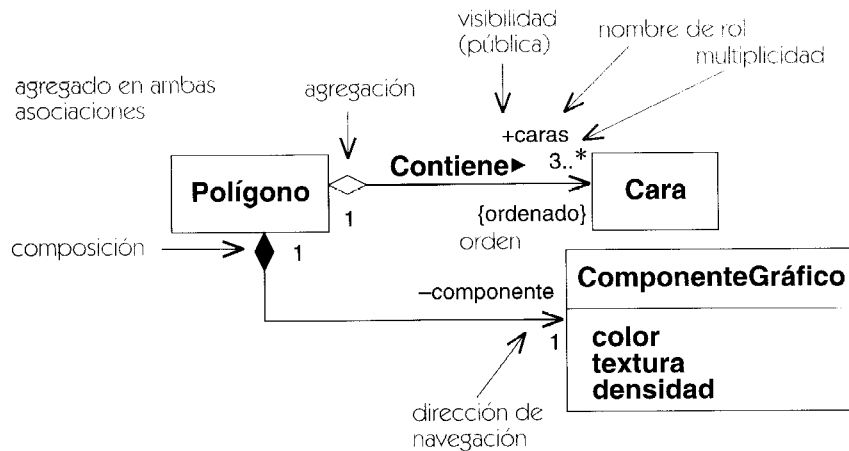


Figura 13.108 Diversos adornos en un extremo de asociación

agregación	Un pequeño rombo hueco en el extremo agregado, relleno si se trata de composición.
alcance destino	El nombre de rol de la clase alcanzada se subraya, en otro caso es alcance de instancia.
calificador	Un pequeño rectángulo entre el extremo de la ruta y la clase fuente. El rectángulo contiene uno o más atributos de la asociación: los calificadores.
especificador de interfaz	Un sufijo que se añade al nombre de rol de la forma : nombre-de-tipo.
multiplicidad	Una etiqueta de texto cerca del extremo de la ruta de la forma min..max.
nombre de rol	Una etiqueta de texto cerca del extremo destino.
ordenación	La propiedad {ordered} cerca del extremo destino significa que hay una lista ordenada de instancias de la clase destino.
navegabilidad	Una punta de flecha en el extremo de la ruta que muestra la dirección de navegación. Si ninguno de los dos extremos tiene flecha significa que la asociación es navegable en ambos sentidos (debido a que realmente son pocas las asociaciones que no son navegables en alguna dirección).
variabilidad	La propiedad {frozen} o {addOnly} cerca del extremo destino. Normalmente se omite cuando es {changeable} pero se puede poner para enfatizar la propiedad.
visibilidad	Un símbolo de visibilidad (+ # -) que precede al nombre de rol.

Si hay más de un adorno en un único rol, se representan siguiendo el siguiente orden, empezando desde el extremo de la ruta más cercano a la clase (Figura 13.16):

calificador

símbolo de agregación o composición

flecha de navegación

Los nombres de rol y la multiplicidad se deberían situar cerca del extremo de la ruta para no confundirlos con los de otra asociación. Pueden ponerse en cualquier lado de la línea. Se ha intentado que se situaran siempre en un mismo lado de la línea, pero esto muchas veces va en detrimento de la claridad. Los nombres de rol y las multiplicidades pueden ir cada una en un lado de la línea o ir juntas (por ejemplo * **empleado**).

Elementos estándar

association, global, local, parameter, self.

fases de modelado

Estados de desarrollo por los que pasa un modelo durante el proceso de diseño y construcción de un sistema.

Véase también proceso de desarrollo.

Discusión

El esfuerzo global de desarrollo se puede dividir en actividades que se centran en diferentes fines. Estas actividades no se realizan en secuencia; más exactamente, se realizan iterativamente durante las fases del proceso de desarrollo. El análisis trata de la captura de requisitos y de la comprensión de las necesidades de un sistema. El diseño trata de inventar un enfoque práctico para el problema, dentro de los límites impuestos por las restricciones de estructuras de datos, algoritmos y partes ya existentes del sistema. La implementación trata de la construcción de la solución en un lenguaje o medio ejecutable (tal como una base de datos o un cierto hardware digital). El despliegue está relacionado con poner en práctica la solución dentro de un entorno físico específico. Estas divisiones son relativamente arbitrarias y no siempre están claras, pero siguen siendo reglas útiles.

Sin embargo, estas vistas del desarrollo no deberían tomarse como fases secuenciales del proceso de desarrollo. En el Proceso en Cascada tradicional se trataban ciertamente como fases distintas. En un proceso de desarrollo iterativo, más moderno, no se trata de fases distintas. En un instante de tiempo dado pueden existir actividades de desarrollo en distintos niveles y quizá lo mejor sea comprenderlas como tareas diferentes que es preciso realizar para cada elemento del sistema, no todas al mismo tiempo.

Piense en un grupo de edificios, cada uno de los cuales posee unos cimientos, paredes y techo; todo ellos tienen que terminarse para todos los edificios, pero no todos a la vez. Normalmente, las partes de cada edificio se irán terminando más o menos por orden. Sin embargo, en algunas ocasiones el tejado puede comenzarse antes de que estén terminadas las paredes. En algunas ocasiones, la distinción entre paredes y techo se difumina: piense en una cúpula geodésica que sale del suelo.

Algunos elementos de UML están destinados a todas las fases del desarrollo. Otros están destinados a propósitos de diseño o implementación y sólo aparecerán cuando el modelo esté suficientemente avanzado. Por ejemplo, las especificaciones de visibilidad para los atributos y operaciones tienden a aparecer en la fase de diseño. En la fase de análisis se incluyen sobre todo miembros públicos.

Discusión

Un proceso de desarrollo en cascada está dividido en fases, cada una de las cuales se realiza para todo el sistema a la vez. Entre las fases tradicionales se cuentan el análisis, diseño, implementación y despliegue. Sin embargo, en un proceso iterativo el desarrollo del sistema no se realiza en formación militar. Los elementos se pueden desarrollar con diferentes velocidades. Sin embargo, cada elemento individual pasa por las mismas fases durante el desarrollo, pero los distintos elementos avanzan con distintas velocidades así que el sistema en conjunto no se encuentra en ninguna fase concreta.

Cada fase de modelado caracteriza a un cierto aspecto del problema que debe ser comprendido y modelado. Las primeras fases capturan las propiedades más lógicas y más abstractas. Las últimas fases están más centradas en la implementación y en el rendimiento. El análisis captura los requisitos y el vocabulario especializado del sistema. El diseño captura los algoritmos y estructuras de datos de la implementación abstraída en unas condiciones idealizadas pero físicamente plausibles. También se preocupa de la eficiencia, de la complejidad computacional y de las consideraciones de ingeniería del software necesarias para construir un sistema admisible. La implementación crea una descripción operativa del sistema en condiciones reales en un lenguaje real de programación, incluyendo las imperfecciones de los lenguajes reales y la descomposición de los artefactos del sistema en partes que se puedan desarrollar y almacenar por separado. El entorno de ejecución se enfrenta a los problemas de concurrencia de recursos, del entorno de computación y del rendimiento a gran escala.

UML contiene toda una gama de estructuras adecuadas para las distintas fases del desarrollo. Algunas estructuras (tales como la asociación y el estado) son significativas en todas las fases. Algunas estructuras (tales como la navegabilidad y la visibilidad) tienen sentido durante el diseño pero representan un detalle de implementación innecesario durante el análisis. Esto no se opone a su definición en una fase temprana del proyecto. Algunas estructuras (tales como la sintaxis específica de un lenguaje de programación) sólo son significativas durante la implementación y son un estorbo para el proceso de desarrollo si se introducen prematuramente.

Los modelos cambian durante el desarrollo. Un modelo de UML adopta formas distintas en cada fase de desarrollo, haciendo hincapié en distintas estructuras de UML. El modelado debería hacerse en la comprensión de que no todas las estructuras son útiles en todas las fases.

Véase proceso de desarrollo para una discusión de las fases de modelado y de las etapas de desarrollo.

fin de un enlace

Denota una instancia de un final de asociación.

flujo

Un tipo de relación que relacione dos versiones del mismo objeto en puntos sucesivos en el tiempo.

Véase también *become*, *copia*.

Semántica

Una relación de flujo relaciona dos versiones del mismo objeto en puntos sucesivos en el tiempo. Puede relacionar dos valores de un objeto en una interacción entre instancias, o puede relacionar dos roles de clasificador que describen el mismo objeto en una interacción entre descriptores.

Representa la transformación de un estado de un objeto en otro. Puede representar un cambio en valor, del estado de control, o de la localización.

Los estereotipos de la dependencia de flujo son *become* y *copy*. Otros estereotipos pueden ser agregados por los usuarios.

Notación

Una relación de flujo se representa como una flecha de líneas discontinuas con la palabra clave de estereotipo apropiada unida. No se puede utilizar una relación de flujo desnuda sin un estereotipo.

Elementos estándar

become, *copy*.

flujo de objeto

Una relación entre los sucesivos puntos de control dentro de una interacción, tal como un grafo de actividades o una colaboración.

Véase también acción, grafo de actividades, colaboración, transición de finalización, mensaje, estado de flujo de objetos, transición.

Semántica

Un gráfico de la vista de interacción representa el flujo de control durante un cómputo. Los elementos primitivos de una interacción son acciones y objetos simples. Un flujo de control representa la relación entre una acción y las acciones de su predecesor y de su sucesor, así como las acciones entre sus objetos de entrada y de salida. De forma abreviada, un flujo de control representa la derivación computacional de un objeto en otro o dos versiones de un objeto en un cierto período de tiempo. (Un flujo de control que implica a un objeto como entrada o salida se llama flujo de objetos.)

Notación

En un diagrama de la colaboración, el flujo de control se representa por mensajes unidos a los roles de la asociación (representando enlaces) que conectan roles de clasificador (representando objetos y otras instancias). En un diagrama de actividades, el flujo de control se muestra con flechas sólidas entre los símbolos de actividad. El flujo del objeto se representa por flechas con línea discontinua entre un símbolo de la actividad o una flecha de flujo de control y un símbolo del estado de flujo del objeto. *Véanse* esas entradas para más detalles.

flujo de objetos

Dícese de cierta variedad de flujo de control que representa la relación entre un objeto y el objeto, operación o transición que lo crea (como resultado) o que utiliza (como entrada).

Véase también flujo de control, estado de flujo de objetos.

Semántica

Un flujo de objetos es una clase de flujo de control que tiene como entrada o salida un estado de flujo de objeto.

Notación

Los flujos de objetos se representan mediante una línea discontinua que va desde la entidad de origen hasta la entidad destino. Una de las entidades (o ambas) pueden ser estados de flujo de control, que se mostrarán como símbolos de objeto. La flecha puede tener una palabra re-

servada para indicar de qué clase de flujo de objetos se trata (become o copy). Si no hay una etiqueta en la flecha, se trata de una relación de conversión.

Véase estado de flujo de objeto, become y copy para ver ejemplos.

foco de control

Un símbolo en un diagrama de secuencia que muestra el período de tiempo durante el cual un objeto está realizando una acción, directamente o a través de un procedimiento subordinado.

Véase activación.

generalización

Una relación taxonómica general entre un elemento más general y un elemento más específico. El elemento más específico es completamente consistente con el elemento más general y contiene información adicional. Una instancia de un elemento más específico puede ser utilizada donde se permita el elemento más general.

Véase también generalización de la asociación, herencia, principio de sustitución, generalización de casos de uso.

Semántica

Una relación de generalización es una relación dirigida entre dos elementos generalizables del mismo tipo, tales como clases, paquetes, u otras clases de elementos. Un elemento se llama padre, y el otro se llama hijo. Para las clases, se llama al padre superclase y al hijo subclase. El padre es la descripción de un conjunto de instancias (indirectas) con las características comunes de todos los hijos; el hijo es una descripción de un subconjunto de esas instancias que tengan las características del padre pero que también tengan propiedades adicionales peculiares al hijo.

La generalización es una relación transitiva, antisimétrica. Una dirección de la relación conduce al padre; la otra dirección conduce al hijo. Un elemento relacionado en la dirección del padre a través de una o más generalizaciones se llama antecesor; un elemento relacionado en la dirección del hijo a través de una o más generalizaciones se llama descendiente. No se permite ninguno ciclo dirigido en la generalización. Una clase no puede tenerse a sí misma por antecesor o descendiente.

En el caso más simple, una clase (o el otro elemento generalizable) tiene un solo padre. En una situación más complicada, un hijo puede tener más de un padre. El hijo hereda la estructura, el comportamiento, y restricciones de todos sus padres. Esto se llama herencia múltiple (puede ser llamado más correctamente generalización múltiple). Un elemento del hijo se refiere a su padre y debe tener visibilidad sobre él.

La generalización se puede aplicar a las asociaciones, así como a los clasificadores, a los estados, a los eventos, y a las colaboraciones.

Para el uso de la generalización a las asociaciones, *véase* la generalización de asociaciones.

Para el uso de la generalización para casos de uso, *véase* la generalización de casos de uso.

Los nodos y los componentes son, en gran medida, como las clases, y la generalización aplicada a ellos se comporta igual que lo hace para con las clases.

Restricciones

Una restricción se puede aplicar a un conjunto de relaciones de generalización y sus hijos que comparten a un padre común. Se pueden especificar las propiedades siguientes.

disjoint	disjunto	Ningún elemento puede tener dos hijos en el conjunto como antecesores (en una situación de herencia múltiple). Ninguna instancia puede ser una instancia directa o indirecta de dos de los hijos (en una semántica múltiple de la clasificación).
overlapping	solapado	Un elemento puede tener dos o más hijos en el conjunto de antecesores. Una instancia puede ser una instancia de dos o más hijos.
complete	completo	Todos los hijos posibles se han enumerado en el conjunto y no puede ser agregado ninguno más.
incomplete	incompleto	No se han enumerado todavía todos los hijos posibles en el conjunto. Se esperan más hijos o se conocen pero no se han declarado todavía.

Notación

La generalización entre clasificadores se representa como una línea continua desde el elemento hijo (tal como una subclase) al elemento de padre (tal como una superclase), con un triángulo hueco grande en el extremo de la línea donde se une el elemento más general (Figura 13.109). Las líneas al padre se pueden combinar para producir un árbol (Figura 13.110).

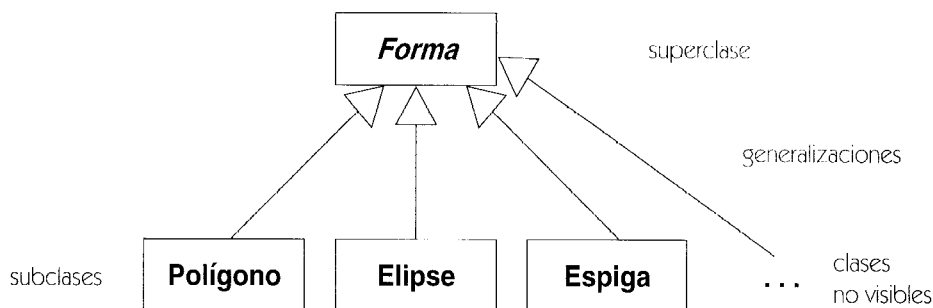


Figura 13.109 Generalización

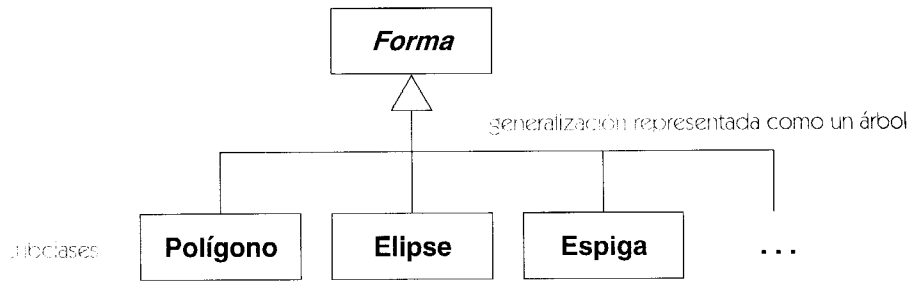


Figura 13.110 Generalización usando un estilo de árbol

La generalización se puede aplicar a las asociaciones, así como a clasificadores, aunque la notación puede ser confusa debido a las líneas múltiples. Una asociación se puede mostrar como clase de asociación con el fin de unir flechas de generalización.

La existencia de clases adicionales en el modelo que no se muestran en un diagrama se puede mostrar usando puntos suspensivos (...) en lugar de una clase. (Note que esto no es una indicación de que pudieran agregarse en el futuro clases adicionales. Indica que existen las clases adicionales ahora pero no se están mostrando. Ésta es una convención de escritura que significa que se ha suprimido la información. No es un elemento semántico.) La presencia de puntos suspensivos (...) como un nodo de subclase de una clase indica que el modelo semántico contiene por lo menos una subclase de la clase que no es visible en el diagrama actual. Los puntos suspensivos pueden tener un discriminador. Este indicador está pensado para ser mantenido automáticamente por una herramienta de edición, no para ser introducido manualmente.

Opciones de representación

Un grupo de líneas de generalización para una superclase dada se puede mostrar como árbol con un segmento compartido (triángulo incluido) a la superclase, ramificando en líneas múltiples a cada subclase. Esto es simplemente un dispositivo notacional. No indica una relación de orden n . En el modelo subyacente, hay una generalización para cada par subclase-superclase. No hay diferencia semántica si los arcos se dibujan por separado.

Si una etiqueta del texto se pone en un triángulo de la generalización compartido por varias líneas de la generalización a las subclases, la etiqueta se aplica a todas las líneas. Es decir, todas las subclases comparten las propiedades dadas.

Ejemplo

La Figura 13.111 muestra la declaración de restricciones en generalizaciones. También ilustra el uso del estilo árbol de la generalización, en el cual las líneas se dibujan en una rejilla ortogonal y comparten una flecha común del padre, así como el estilo binario, en el cual cada relación padre-hijo tiene su propia flecha oblicua.



Figura 13.111 Restricciones de generalización

Discusión

El elemento del padre en una relación de generalización se puede definir sin el conocimiento de los hijos, pero los hijos deben saber generalmente la estructura de sus padres para trabajar correctamente. En muchos casos, sin embargo, se diseñan para ser extendidos por los hijos y se incluye al padre cierto conocimiento de los hijos previstos. Una de las glorias de la generalización, sin embargo, es que se descubre que los nuevos hijos a menudo no habían sido anticipados por el diseñador del elemento padre, conduciendo a una expansión que es compatible en esencia con la intención original.

La relación de realización es como una generalización en la cual solamente se hereda la especificación del comportamiento en vez de la estructura o implementación. Si el elemento de la especificación es una clase abstracta sin atributos ni asociación, y con operaciones solamente abstractas, entonces la generalización y la realización son casi equivalentes, ya que no hay nada que heredar excepto la especificación del comportamiento. Sin embargo, observe que la realización no describe realmente al cliente; por lo tanto, las operaciones deben estar en el cliente o heredadas de algún otro elemento.

Elementos estándar

complete, disjoint, implementation, incomplete, overlapping.

generalización de asociaciones

Relación de generalización entre dos asociaciones.

Véase también asociación, generalización.

Está permitida la generalización entre asociaciones, aunque es poco frecuente. Como en toda relación de generalización, el elemento hijo debe añadir algo a la declaración (reglas de definición) del padre y debe definir un subconjunto del conjunto de instancias del padre. Añadir algo a la declaración significa añadir restricciones adicionales. Una asociación hija es más restringida que su padre. Por ejemplo, en la Figura 13.112, si la asociación padre conecta las clases **Tema** y **Símbolo**, entonces una asociación hija podría conectar las clases **Orden** y **SímboloOrden**, donde **Orden** es hija de **Tema** y **SímboloOrden** hija de **Símbolo**. Ser un subconjunto del conjunto de instancias del padre significa que todos los enlaces de la asociación hija son también enlaces de la asociación padre, pero no a la inversa. El ejemplo sigue esta regla. Un enlace que conecte **Orden** y **SímboloOrden** conectará también **Tema** y **Símbolo**, pero no todos los enlaces que conecten **Tema** y **Símbolo** conectarán necesariamente **Orden** y **SímboloOrden**.

La asociación hija se conectará con la asociación padre utilizando un símbolo de generalización (una flecha con línea continua y punta triangular hueca). La punta de la flecha estará en la asociación padre. Debido a que las líneas conectan otras líneas, la notación de generalización de asociaciones puede ser confusa y debe usarse con cuidado.

La Figura 13.112 muestra dos especializaciones de la asociación general **modelo-vista** entre **Tema** y **Símbolo**: la asociación entre **Cliente** y **SímboloCliente** y la asociación **Pedido** y **SímboloPedido**. Cada una de ellas conecta una clase **Tema** con una clase **Símbolo**. La asociación general **Tema-Símbolo** puede verse como una asociación abstracta en la cual las dos asociaciones hijas son concretas.

Este diagrama ilustra la generalización de asociación entre los conceptos **Tema** y **Símbolo**. Se muestran las relaciones de generalización y asociación entre los siguientes elementos:

- Tema** y **Símbolo**: Se relacionan mediante una asociación etiquetada como "modelo".
- Pedido** y **Símbolo**: Se relacionan mediante una asociación etiquetada como "vista".
- Cliente** y **Símbolo**: Se relacionan mediante una asociación etiquetada como "vista".
- Pedido** y **Símbolo**: Se relacionan mediante una asociación etiquetada como "vista".
- Cliente** y **Símbolo**: Se relacionan mediante una asociación etiquetada como "vista".
- Pedido** y **Símbolo**: Se relacionan mediante una asociación etiquetada como "vista".
- Cliente** y **Símbolo**: Se relacionan mediante una asociación etiquetada como "vista".

Las flechas indican la dirección de la generalización de asociación, mostrando cómo los conceptos más específicos se relacionan con los más generales.

Figura 13.112 Generalización de asociaciones

Elementos estándar

destroyed.

generalización de casos de uso

Una relación taxonómica entre un caso de uso (el hijo) y el caso de uso (el padre) que escribe las características que comparte el hijo con otros casos de uso que tengan el mismo padre. Se trata de una generalización aplicable a los casos de uso.

Semántica

Un caso de uso padre puede especializarse en uno o más casos de uso hijos que representan formas más específicas del padre (Figura 13.113). Un hijo hereda todos los atributos, operaciones y relaciones de su padre, porque un caso de uso es un clasificador. La implementación de una operación heredada puede anularse en una colaboración que realice algún caso de uso hijo.



Figura 13.113 Generalización de caso de uso

El hijo hereda la secuencia de comportamiento del padre y puede insertar en ella algún comportamiento adicional (Figura 13.114). El padre y el hijo son potencialmente instanciables.

Comportamiento de caso de uso para el padre, **Verificar Identidad**:

El padre es abstracto, no hay secuencia de comportamiento.

Un descendiente concreto tiene que proporcionar el comportamiento, según se muestra más abajo.

Comportamiento de caso de uso para el hijo, **Comprobar Contraseña**:

- Obtener contraseña en base de datos maestra
- Pedir contraseña al usuario
- El usuario proporciona la contraseña
- Comparar contraseña con entrada del usuario

Comportamiento de caso de uso para el hijo, **Exploración de retina**:

- Obtener signature retinal en base de datos maestra
- Explorar la retina del usuario y obtener su signature
- Comparar la signature maestra con la signature explorada

Figura 13.114 Secuencias de comportamiento para casos de uso hijo y padre

(si no son abstractos) y las diferentes especializaciones del mismo padre son independientes, a diferencia de una relación de extensión en la cual las extensiones múltiples modifican implícitamente un mismo caso de uso base. Es posible agregar comportamiento al caso de uso hijo añadiendo pasos a la secuencia de comportamiento heredada del padre, así como declarando relaciones de extensión y de inclusión para el hijo. Si el padre es abstracto, su secuencia de comportamiento puede tener secciones que sean explícitamente incompletas en el padre y que debe proporcionar el hijo. El hijo puede modificar pasos heredados del padre, pero tal como sucede al redefinir métodos, es preciso utilizar con cuidado esta capacidad porque debe mantenerse la intencionalidad del padre.

La relación de generalización conecta un caso de uso hijo con un caso de uso padre. Un caso de uso padre puede acceder a los atributos definidos por el caso de uso padre; puede incluso modificarlos.

La capacidad de sustitución para los casos de uso significa que la secuencia de comportamiento de un caso de uso hijo tiene que incluir la secuencia de comportamiento de su padre. Sin embargo, no tienen por qué ser contiguos los pasos en la secuencia del padre; el hijo puede intercalar pasos adicionales entre los pasos de la secuencia de comportamiento heredados del padre.

El uso de herencia múltiple en casos de uso requiere la especificación explícita de la forma en que se intercalan las secuencias de comportamiento de los padres para crear la secuencia correspondiente al hijo.

La generalización de casos de uso puede hacer uso de la herencia privada para compartir la implementación de un caso de uso base sin tener una capacidad de sustitución completa, pero esta capacidad debe emplearse con parquedad.

Notación

Se emplea el símbolo normal de generalización —una línea continua que va del hijo al padre con un cabeza de flecha triangular hueca en la línea que toca al símbolo del padre.

Ejemplo

La Figura 13.113 muestra el caso de uso abstracto **Verificar Identidad** y su especialización como dos casos de uso concretos, cuyo comportamiento se muestra en la Figura 13.114.

grafo de actividades

Caso especial de una máquina de estados en el cual todos o la mayoría de los estados son estados de actividad o estados de acción, y en el que la mayoría de las transiciones son disparadas por la finalización de una actividad en los estados origen. Un grafo de actividades muestra un procedimiento o un flujo de trabajo. Un grafo de actividades es una unidad completa en el modelo, mientras que un diagrama de actividades es un diagrama que muestra un grafo de actividades.

Véase también máquina de estados.

Semántica

Un grafo de actividades es una máquina de estados que enfatiza los pasos secuenciales y concurrentes de un procedimiento de computación. Los flujos de trabajo son ejemplos de procedimientos que a menudo se modelan utilizando grafos de actividades. Los grafos de actividades generalmente aparecen en las primeras etapas del diseño, antes de que se tomen las decisiones de implementación, y en particular, antes de que se les hayan asignado a los objetos todas las tareas a realizar. Este tipo de grafo es una variante de una máquina de estados en la cual un estado representa la realización de una actividad, como un cómputo o una operación continua del mundo real, y las transiciones se disparan como consecuencia de la finalización de operaciones. Un grafo de actividades puede asociarse a la implementación de una operación o de un caso de uso.

En un grafo de actividades los estados son principalmente estados de actividad o estados de acción. Un estado de actividad representa un estado con un cómputo interno y al menos una transición de finalización saliente que se dispara cuando finaliza la actividad del estado. Puede haber varias transiciones de salida si tienen condiciones de guarda. Los estados de actividad no deberían tener transiciones internas o transiciones de salida basadas en eventos explícitos: utilice estados normales en estas situaciones. Un estado de acción es un estado atómico, es decir, un estado que no puede ser interrumpido por transiciones, ni siquiera aquéllas de sus estados adyacentes.

El uso típico de un estado de actividad es modelar un paso en la ejecución de un procedimiento. Si todos los estados de un modelo son estados de actividad, entonces el resultado de la computación no dependerá de eventos externos. La computación es determinista si las actividades concurrentes no acceden a los mismos objetos y los tiempos relativos de finalización de las actividades concurrentes no afectan a los resultados.

Los grafos de actividades pueden incluir estados de espera ordinarios cuya existencia se dispara gracias a eventos, pero este uso frustra el propósito de centrar la atención en las actividades: utilice un modelo de estados ordinario si tiene más de unos pocos estados ordinarios.

Concurrencia Dinámica. Un estado de actividad con concurrencia dinámica representa la ejecución concurrente de múltiples cómputos independientes. La actividad se invoca con un conjunto de listas de argumentos, de forma que cada miembro del conjunto es la lista de argumentos de una invocación a la actividad. Las invocaciones son independientes unas de otras. Cuando se han completado todas las invocaciones la actividad finaliza y dispara su transición de finalización.

Flujo de objetos. A veces es útil observar las relaciones entre una operación y aquellos objetos que recibe como argumentos o que devuelve como resultado. La entrada de una operación y la salida de la misma se pueden mostrar como un estado de flujo de objetos, lo que constituye un estereotipo de un estado que representa la existencia de un objeto de una clase dada en un punto concreto del cómputo. Para mayor precisión, se puede declarar que el objeto de entrada o de salida esté en un estado determinado dentro de su clase. Por ejemplo, la salida de una operación “firmar contrato” será un estado de flujo de objetos de la clase Contrato en el estado “firmado”. Este estado de flujo de objetos puede ser la entrada de muchas otras operaciones.

Calles. Las actividades de un grafo de actividades se pueden separar en particiones según diversos criterios. Cada partición representará una división significativa de las responsabilidades de las actividades, por ejemplo, las responsabilidades de negocio de la organización en un paso de un flujo de trabajo dado. A las particiones se les llama calles debido a la notación gráfica utilizada para su representación.

Notación

Un grafo de actividades se representa mediante un diagrama de actividades. Los grafos de actividades son una variante de la máquina de estados, pero hay algunos aspectos de la notación que son particularmente aplicables a los diagramas de actividades: estados de actividad, bifurcaciones, uniones, calles, estados de flujo de objetos, clases en un estado, notación de recepción y envío de señales y eventos diferidos. Véanse estas entradas para más detalles.

Véase iconos de control para más información sobre algunos símbolos opcionales que pueden ser útiles en los diagramas de actividades.

Ejemplo

La Figura 13.115 muestra un flujo de trabajo de las actividades a realizar en el procesamiento de un pedido en una taquilla de teatro. Incluye una bifurcación y la subsiguiente unión basada

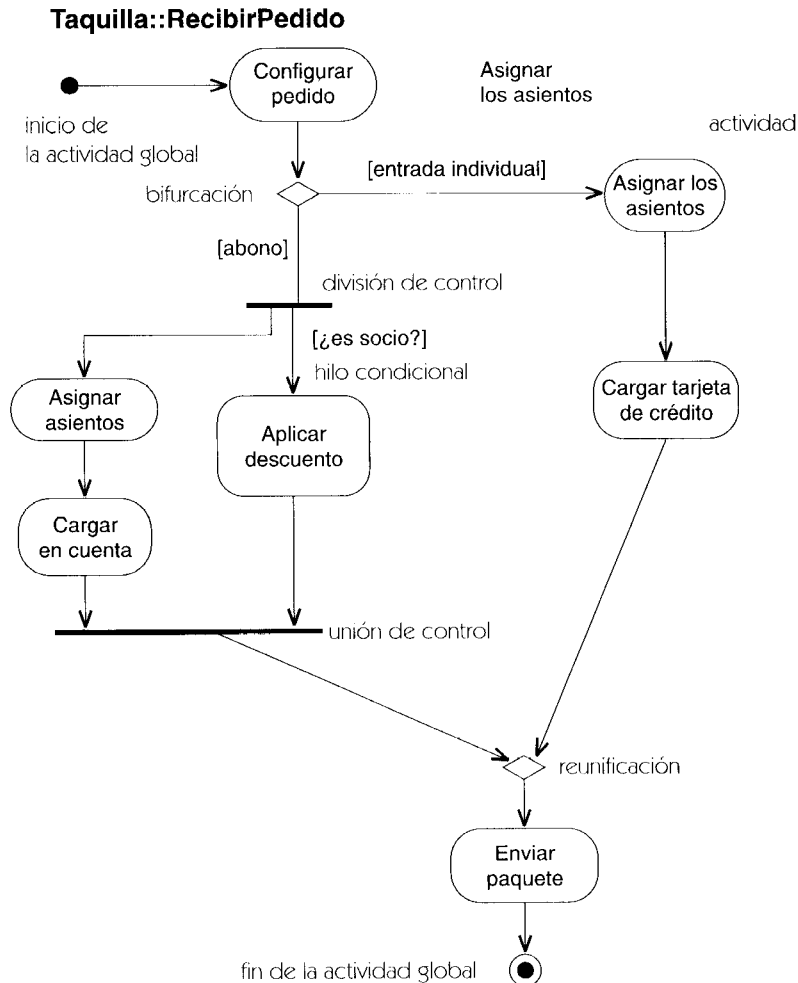


Figura 13.115 Diagrama de actividades

en si el pedido es para un abono o para entradas individuales. La división inicia las actividades concurrentes que lógicamente se producen al mismo tiempo, y cuya ejecución real puede o no solaparse en el tiempo. La concurrencia termina en una unión al mismo nivel. Si sólo hay una persona involucrada, las actividades concurrentes pueden realizarse en cualquier orden (suponiendo que no puedan realizarse a la vez, algo que permite el modelo pero que es difícil en la práctica). Por ejemplo, el encargado de la taquilla podría asignar los asientos, luego aplicar el descuento y luego cargar el importe en la cuenta; o aplicar el descuento, luego asignar los asientos y por último cargar el importe en la cuenta, pero no debería cargar el importe sin antes haber asignado los asientos.

Un segmento de salida de la división tiene una condición de guarda que comprueba si el abonado es socio. Esto es un hilo condicional que se ejecuta sólo si se satisface la condición de guarda. Si no se ejecuta el hilo, el segmento de entrada al siguiente punto de unión se considera completado. Si el abonado no es socio sólo se inicia un hilo, ya que se le asignan asientos y se le carga el importe en su cuenta pero no se le hace descuento.

Calles. Las actividades de un grafo de actividades se pueden separar en regiones a las que se llama calles por su representación gráfica, ya que se dibujan como regiones diferentes de un diagrama separadas por líneas discontinuas. Una calle sirve para organizar los contenidos de un grafo de actividades. No tiene un significado inherente, por lo que se puede utilizar como lo desee el que realiza el modelado. A menudo cada calle representa una unidad organizativa dentro de una organización del mundo real.

Ejemplo

En la Figura 13.116 las actividades están divididas en tres particiones mediante calles, cada una de las cuales corresponde a un usuario. UML no establece ningún requisito sobre si las particiones deben corresponderse con objetos, aunque en este ejemplo hay clases obvias que encajarían perfectamente en cada partición, y esas clases serían las únicas que realizarían las operaciones para implementar cada actividad en el modelo final.

La figura también muestra el uso de símbolos de flujo de objetos. Los flujos de objetos corresponden a los diferentes estados por los que pasa un objeto Pedido a través de toda la red de actividades que realiza. El símbolo **Pedido[ubicado]**, por ejemplo, significa que en ese lugar del cómputo el pedido ha avanzado hasta el estado **ubicado** en la actividad **Solicitar-Servicio** pero aún no ha sido consumido por la actividad **RecogerPedido**. Cuando se completa la actividad **RecogerPedido**, el pedido pasa al estado **introducido**, como se muestra en el diagrama mediante el símbolo de flujo de objetos situado en la salida de la actividad **RecogerPedido**. Todos los flujos de objetos de este ejemplo representan el mismo objeto en momentos diferentes de su vida, y precisamente por representar el mismo objeto, no pueden existir al mismo tiempo. Se puede dibujar entre ellos una ruta de control secuencial, como se ve en el diagrama.

Flujo de objetos. Los objetos que son salida o entrada de una acción se pueden representar mediante símbolos de objeto. El símbolo representa al objeto en el momento del cómputo en el que se produce la entrada o en el que se realiza la salida (normalmente un objeto hace las dos cosas). Se dibuja una flecha discontinua que va desde la transición de salida de un estado de actividad hasta un flujo de objetos que es una de sus salidas, y otra flecha discontinua desde el

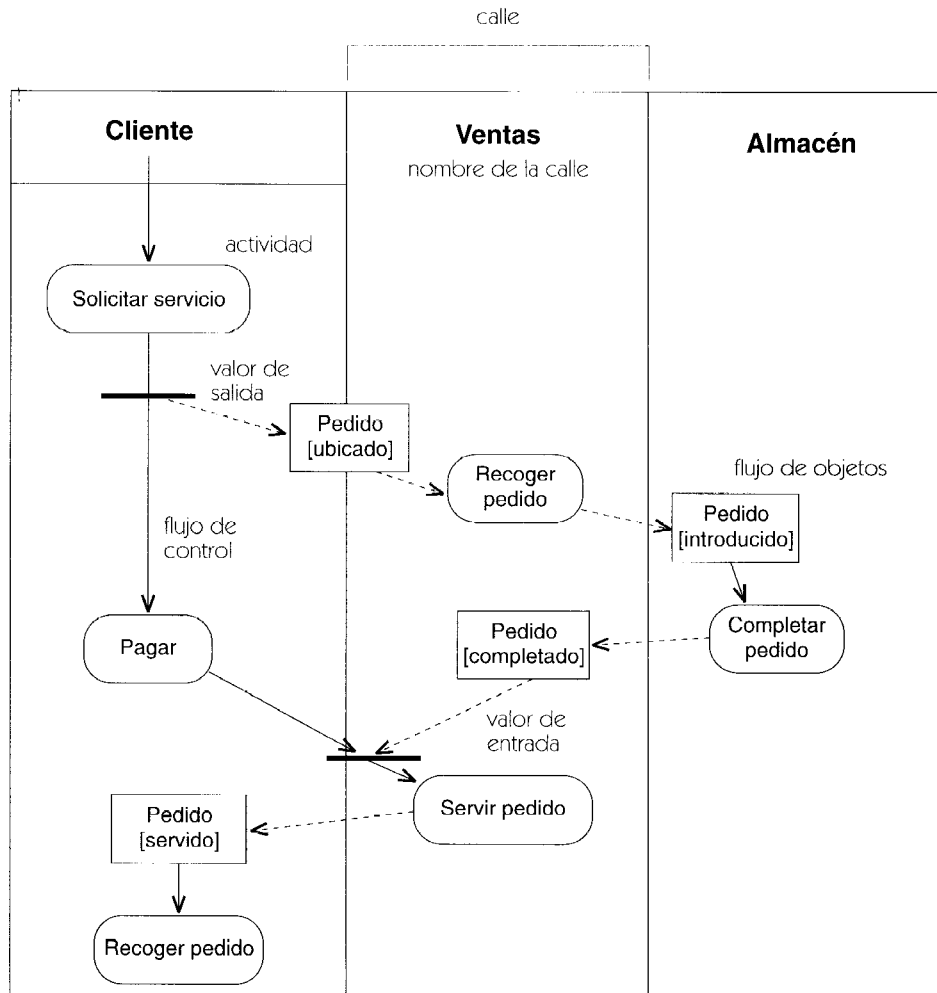


Figura 13.116 Diagrama de actividades con calles

flujo de objetos hasta la transición de entrada de un estado de actividad que utiliza el objeto como entrada. El mismo objeto puede ser, y normalmente es, salida de una actividad y entrada de una o más actividades posteriores.

Las flechas de flujo de control (continuas) pueden omitirse cuando las flechas de flujo de objetos (discontinuas) proporcionan una restricción redundante. En otras palabras: cuando una acción produce una salida que es entrada de una acción posterior, la relación de flujo de objetos implica una restricción de control.

Véase estado de flujo de objetos.

Clase en un estado. Es frecuente que unas cuantas actividades sucesivas manipulen el mismo objeto para cambiar su estado. Precisando aún más, se puede representar el objeto muchas veces en un diagrama, de forma que cada una de las apariciones del mismo objeto represente un estado diferente a lo largo de su vida. Para distinguir entre las distintas apariciones de un mismo objeto, el estado del objeto en cada punto se puede poner entre corchetes a la

derecha del nombre de la clase, por ejemplo, **PeticiónCompra[aprobado]**. Esta notación puede utilizarse también en diagramas de colaboración.

Véase también iconos de control, donde se muestran otros símbolos que se pueden utilizar en los diagramas de actividades.

Eventos diferidos. A veces existe un evento cuya existencia debe ser pospuesta para más tarde mientras se ejecuta alguna otra actividad (normalmente un evento que no se trata inmediatamente, se pierde). Un evento diferido es un evento que se sitúa en una cola interna hasta que se utiliza o hasta que es descartado. Cada estado o actividad pueden especificar el conjunto de eventos que serán diferidos si se producen durante el estado o la actividad. Los demás eventos se tratarán inmediatamente para no perderlos. Cuando la máquina de estados entra en un nuevo estado, se producen todos los eventos diferidos a menos que el nuevo estado también los marque como diferidos. Si un evento que fue diferido en el estado anterior dispara una transición del nuevo estado, la transición se ejecuta inmediatamente. Si son varias las transiciones potenciales, queda indefinido cuál de ellas se ejecutará; si se impone una regla para seleccionar una se estará introduciendo una pequeña variación de la semántica.

Si se produce un evento durante un estado para el cual está diferido y dicho evento dispara una transición, no se situaría en la cola. Si no dispara ninguna transición, se sitúa en la cola. Si cambia el estado activo, los eventos de la cola pueden disparar transiciones en el nuevo estado pero permanecerán en la cola si siguen siendo eventos diferidos en el nuevo estado. La capacidad de quitarle la marca de diferido a un evento para que dispare una transición, es útil sobre todo en un estado compuesto donde el evento debe ser diferido pero que a su vez debe permitir transiciones en uno o más subestados. A parte de eso, tendría que ser diferido en cada uno de los subestados en los que no dispara transiciones. Obsérvese que si un evento diferido debe disparar una transición pero no se cumple la condición de guarda, el evento no ha disparado la transición y por tanto no puede eliminarse de la cola.

Gráficamente, un evento que puede diferirse se lista dentro del estado, seguido por una barra inclinada y la operación especial **defer** (diferir). Si se produce el evento y no dispara ninguna transición, se almacena y vuelve a producirse cuando el objeto inicie una transición hacia otro estado, y en ese momento puede ser diferido de nuevo. Cuando el objeto llega a un estado en el que el evento no está diferido, debe ser aceptado o si no será ignorado. La ausencia de una sentencia **defer** tiene el efecto de cancelar una marca de diferido previa. La indicación **defer** se puede poner en un estado compuesto, en cuyo caso el evento es diferido a lo largo de todo el estado compuesto.

Los estados de acción son atómicos, por lo que difieren implícitamente todos los eventos que ocurren durante el tiempo en que están activos, por lo que no es necesario marcarlos como diferidos. Cualquier evento que se produzca pasa a diferido hasta que se complete la acción, momento a partir del cual el evento podría disparar una transición.

Ejemplo

La Figura 13.117 muestra los pasos necesarios para hacer café. En este ejemplo, no se muestra el objeto externo (**cafetera**) sino sólo las actividades realizadas directamente por la persona que va a hacer el café. El acto de encender la cafetera se modela como un evento que se envía a la cafetera. La actividad **Tomar Tazas** ocurre después de encender la cafetera. Después de tomar las tazas hay que esperar a que se apague la luz de la cafetera. Sin embargo, hay un problema.

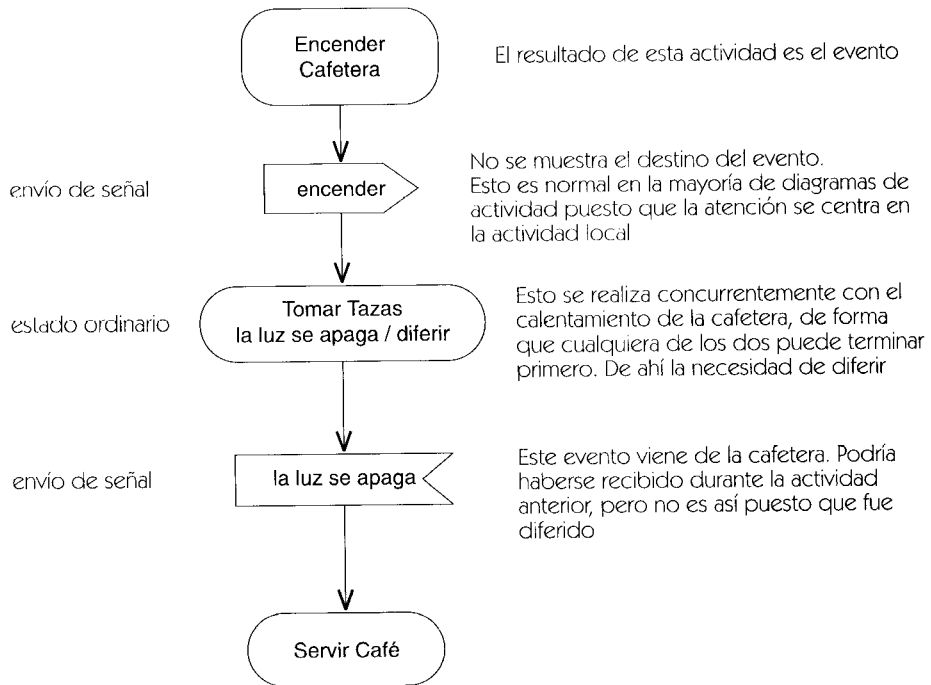


Figura 13.117 Eventos diferidos e iconos de control

Si el evento **la luz se apaga** se produce antes de que termine la actividad **Tomar Tazas**, el evento se pierde porque la máquina de estados aún no está preparada para tratarlo. Para evitar que se pierda un evento, en el estado de actividad **Tomar Tazas** se ha marcado como diferido el evento **la luz se apaga**. Si el evento se produce mientras se está ejecutando la actividad, no se perderá sino que se almacenará en una cola de espera hasta que la máquina de estados abandone el estado **Tomar Tazas**, momento en el cual se procesará y luego se disparará la transición.

Obsérvese que el evento **la luz se apaga** no es un disparador del estado **Tomar Tazas**, y sin embargo si se produce la actividad no termina. El evento es un disparador del estado de recepción de señal inmediatamente posterior a la finalización de la actividad **Tomar Tazas**.

Concurrencia dinámica. La concurrencia dinámica con un valor distinto de uno se representa mediante una cadena de multiplicidad en la parte superior derecha del símbolo de la actividad (Figura 13.118). Esto indica que las múltiples copias de la actividad se ejecutan concurrentemente. La actividad dinámica recibe un conjunto de listas de argumentos. Los detalles se deben describir mediante texto. Si se aplica la concurrencia a varias actividades, debe encerrarse en una actividad compuesta que lleve el indicador de multiplicidad.

Estructura de grafo. Una máquina de estados debe estar bien anidada, es decir, particionada recursivamente en subestados concurrentes o secuenciales. En un grafo de actividades, las bifurcaciones y divisiones deben estar bien anidadas. Cada bifurcación debe tener su correspondiente reunificación, y cada división su correspondiente unión, pero eso no siempre es lo más conveniente en un grafo de actividades. Es bastante común encontrarse con un grafo parcialmente ordenado, en el cual no hay grafos dirigidos pero las bifurcaciones y divisiones no coinciden necesariamente. Un grafo así no está bien anidado, pero puede transformarse en un

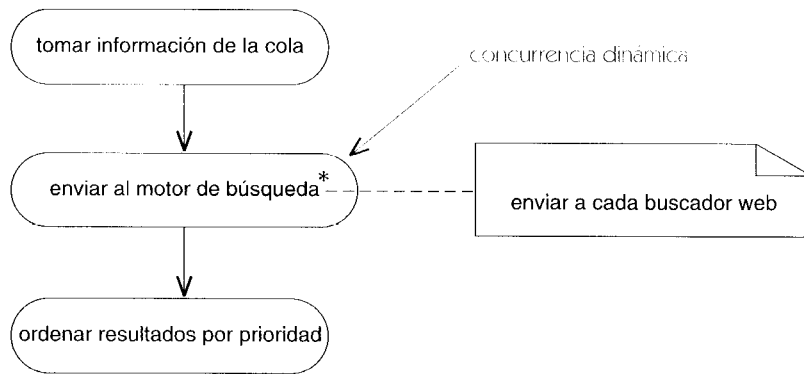


Figura 13.118 Concurrencia dinámica

grafo bien anidado asignando actividades a hilos e introduciendo estados de sincronización cuando las transiciones crucen la frontera de los hilos. La descomposición no es única, pero en un grafo parcialmente ordenado todas las descomposiciones posibles tienen la misma semántica, y por lo tanto no es necesario representar una descomposición explícita o estados de sincronización en un grafo de este tipo. En grafos más complicados que contienen condiciones y concurrencia podría ser necesario ser más explícito sobre la descomposición.

herencia

Mecanismo mediante el cual unos elementos más específicos incorporan la estructura y el comportamiento definidos por otros elementos más generales.

Véase también descriptor completo, generalización.

Semántica

La herencia permite construir una descripción completa de un elemento generalizable ensamblando fragmentos de declaraciones a partir de una jerarquía de generalización. Una jerarquía de generalización es un árbol (o más exactamente, un orden parcial) de declaraciones de elementos de un modelo, tales como clases. Sin embargo, las declaraciones no son declaraciones de un elemento completo y utilizable. Cada declaración es en realidad una declaración incremental que describe lo que añade el elemento de declaración a las declaraciones de sus antecesores dentro de la jerarquía de generalización. La herencia es el proceso (implícito) de combinación de esas declaraciones incrementales para formar descriptores completos que describen instancias reales.

Se puede pensar que todo elemento generalizable posee dos descripciones: una declaración de segmento y un descriptor completo. La declaración de segmento es la lista incremental de características que declara el elemento en el modelo: los atributos y operaciones que declara una clase, por ejemplo. La declaración de segmento es la diferencia entre el elemento y sus predecesores. El descriptor completo es la unión de los contenidos de las declaraciones de segmento que aparecen en un segmento y en todos sus antecesores.

Esto es la herencia. Se trata de la declaración incremental de un elemento. Hay otros detalles, tales como los algoritmos de búsqueda de métodos, las *vtables* y demás, que son meramente mecanismos de implementación para hacer que funcione en un determinado lenguaje: no forma parte de la definición esencial. Aunque esta descripción pueda parecer extraña a primera vista, está libre de los estorbos de implementación que se encuentran en casi cualquier otra definición, pero es compatible con ellas.

Conflictos

Si hay una misma característica que aparece más de una vez entre el conjunto de segmentos heredados, quizá se produzca un conflicto. No se puede declarar ningún atributo más de una vez en un conjunto heredado. Si esto sucediera, las declaraciones entran en conflicto y el modelo está mal formado. (Esta restricción no es esencial por razones lógicas. Se ha impuesto para evitar cierta confusión que podría producirse si fuera preciso distinguir los atributos mediante rutas de acceso.)

Se podría declarar dos veces la misma operación, siempre y cuando la declaración fuera exactamente la misma (sin embargo, los métodos pueden ser distintos) o bien una declaración descendiente fortalece una declaración heredada (por ejemplo, declarando que un descendiente es una consulta, o incrementando su estado de concurrencia). La declaración de un método en un descendiente sustituye (anula) a la declaración de un método en su antecesor. No hay conflicto. Si se heredan métodos distintos de dos antecesores diferentes que no poseen, a su vez, una relación de antecesor, entonces los métodos entran en conflicto y el modelo está mal formado.

Discusión

La generalización es una relación taxonómica entre elementos. Describe lo que es un elemento. La herencia es un mecanismo para combinar descripciones incrementales compartidas, con objeto de formar una descripción completa de un elemento. No son una misma cosa, aunque están íntimamente relacionadas. El mecanismo de herencia, aplicado a la relación de generalización, permite factorizar y compartir las descripciones y el comportamiento polimórfico. Éste es el enfoque que adopta la mayoría de los lenguajes orientados a objetos y también UML. Pero hay que tener en cuenta que existen otros enfoques que podrían haberse adoptado y que emplean algunos lenguajes de programación.

herencia de implementación

La herencia de la implementación de un elemento predecesor, esto es, de su estructura (tal como los atributos y operaciones) y de su código (tal como sus métodos). Por contraste, la herencia de interfaz implica la herencia de especificaciones de interfaz (esto es, de operaciones) pero no de métodos ni de estructuras de datos (atributos y asociaciones).

El significado normal de la generalización en UML incluye la herencia *tanto* de interfaz como de implementación. Para heredar *solamente* la implementación (herencia privada) se aplica el estereotipo «**implementation**» a una generalización. Para admitir una interfaz sin

comprometerse con una implementación, se le aplica una relación de realización a una interfaz.

Véase también generalización, herencia, herencia de interfaz, herencia privada.

herencia de interfaz

Denota la herencia de la interfaz de un elemento predecesor, pero no de su implementación ni de su estructura de datos. La intención de admitir una interfaz sin comprometerse con una implementación se modela empleando la realización.

Obsérvese que en UML la generalización implica la herencia *tanto* de la interfaz como de la implementación. Para heredar solamente la implementación sin la interfaz, utilice la herencia privada.

herencia múltiple

Denota un punto de generalización de variación semántica en el cual un elemento puede tener más de un predecesor. Se trata de la suposición por defecto en UML y se necesita para el correcto modelado de muchas situaciones, aun cuando los modeladores pueden optar por restringir su utilización para ciertas clases de elementos. Por contraste: herencia simple.

herencia privada

Denota la estructura de herencia en una situación tal que no se hereda la especificación de comportamiento.

Véase también herencia de implementación, herencia de interfaz, principio de capacidad de sustitución.

Semántica

Una generalización puede tener el estereotipo «**implementation**». Esto indica que el elemento cliente (normalmente una clase) hereda la estructura (atributos, asociaciones y operaciones) del elemento proveedor, pero no pone necesariamente esta estructura a disposición de sus propios clientes. Dado que los antecesores de esta clase (u otro elemento) no son visibles para las otras clases, no se puede utilizar una instancia de la clase como variable o parámetro declarado en la clase proveedora. En otras palabras, la clase no se puede sustituir por sus proveedores heredados de forma privada. La herencia privada no sigue el principio de capacidad de sustitución.

Notación

Se denota la herencia privada mediante la palabra reservada «**implementation**» junto a una flecha de generalización procedente del elemento que hereda (el cliente) y que llega al elemento que proporciona la estructura que hay que heredar (el proveedor).

Discusión

La herencia privada es simplemente un mecanismo de implementación y no debería pensarse que se trata de una utilización de la generalización. La generalización requiere la capacidad de sustitución. La herencia privada no es especialmente significativa en un modelo de análisis, que no implica la estructura de implementación. Incluso para la implementación, debería utilizarse con cuidado porque implica una utilización no semántica de la herencia. Suele ser una alternativa más limpia emplear una asociación con la clase proveedora. Hay muchos autores (incluyendo alguno de los presentes) que afirman que no debería emplearse nunca porque hace uso de la herencia de una manera no semántica que resulta peligrosa cuando se hacen cambios en el modelo.

herencia simple

Variación semántica de la generalización en la que un elemento sólo puede tener un padre. Admitir la herencia simple o la herencia múltiple es un punto de variación semántica.

hijo/a

Elemento más específico de una relación de generalización. Cuando se trata de clases se denomina subclase. Una cadena de una o más relaciones hijas se denomina descendiente. Antónimo: padre.

Semántica

Un elemento hijo hereda las características de su padre (e indirectamente las de sus antecesores) y puede declarar características adicionales propias. También hereda todas las asociaciones y restricciones en las que participan sus antecesores. Un elemento hijo cumple el principio de capacidad de sustitución, esto es, una instancia de un descriptor satisface cualquier declaración de variable clasificada como uno de los antecesores del descriptor. Una instancia de un hijo es una instancia indirecta del padre.

hilo

(De *hilo de control*) Una única ruta de ejecución que recorre un programa, modelo dinámico u otra representación de flujo de control. También es un estereotipo para la implementación de un objeto activo como proceso ligero.

Véase objeto activo, transición compleja, estado compuesto, máquina de estados, estado de sincronización.

hilo condicional

Una región del grafo de actividades iniciado por un segmento protegido de salida de una división que termina con un segmento de entrada a la unión correspondiente.

Véase estado compuesto, transición compleja.

hipervínculo

Una conexión invisible entre dos elementos de notación, que puede ser recorrida por algún comando.

Véase también diagrama.

Notación

Una notación escrita en un trozo de papel no contiene información oculta. Sin embargo, una notación en una pantalla de computador puede contener hipervínculos adicionales invisibles que no resultarán visibles desde un punto de vista estático pero pueden invocarse dinámicamente para acceder a alguna otra información, bien en una vista gráfica o en una tabla de texto. Estos enlaces dinámicos forman parte de una notación *dinámica* en idéntica proporción que la información visible, pero este documento no prescribe su forma. Los consideramos como una responsabilidad de la herramienta. Este documento pretende definir una notación *estática* para UML, bien entendido que es posible que una parte útil e interesante de la información no aparezca claramente (o no aparezca de ninguna forma) en una de estas vistas. Por otra parte, no se puede disponer de un conocimiento adecuado para especificar el comportamiento de todas las herramientas dinámicas y tampoco se pretende asfixiar la innovación en las presentaciones dinámicas. Eventualmente, es posible que algunas notaciones dinámicas queden tan firmemente establecidas que sea posible llegar a un estándar, pero no creemos que esto deba hacerse en estos momentos.

hoja

Término que denota un elemento generalizable carente de descendientes en la jerarquía de generalización. Tiene que ser completo (estará completamente implementado) para que pueda servir para algo.

Véase también abstracto, concreto.

Semántica

La propiedad de ser una hoja declara que un elemento *es* una hoja. El modelo estará mal formado si se declara un descendiente de uno de estos elementos. El propósito es garantizar que una clase no pueda ser modificada, por ejemplo, porque el comportamiento de la clase tienen que estar bien establecido para que sea fiable. La declaración de una hoja permite además la compilación por separado de partes de un sistema, asegurando que los métodos no se pueden redefinir y permitiendo copiar el código de esos métodos en vez de realizar llamadas a los mismos (*code inlining*, para la mejora de la velocidad de ejecución). Un elemento en que esta propiedad sea falsa puede ser ciertamente una hoja pero podría tener descendientes en el futuro si se modifica el modelo. Ser una hoja o estar obligado a ser una hoja no son propiedades semánticas fundamentales.

iconos de control

Símbolos opcionales que proporcionan una conveniente notación rápida para los diferentes patrones del control.

Véase también grafo de actividades, colaboración, máquina de estados.

Notación

Los siguientes símbolos están pensados para su uso en diagramas de actividad, pero pueden también ser utilizados en diagramas de estados, si se quiere. Estos símbolos no permiten nada que no se pudiera mostrar usando los símbolos básicos, pero pueden ser convenientes para ciertos patrones de control comunes.

Bifurcación. Una bifurcación es un conjunto de transiciones que salen de un estado simple de tal forma que siempre es satisfecha exactamente una de las condiciones de guarda de una de las transiciones. Es decir, si ocurre el evento disparador, sólo puede dispararse una transacción. Las condiciones de guarda esencialmente representan una bifurcación del control. Si la transición es una transición de terminación, entonces una bifurcación es una decisión pura. Por conveniencia, una salida de la bifurcación se puede etiquetar con la palabra clave **else**. Se toma esta ruta cuando no es posible tomar ningún otro.

Una bifurcación se denota como un diamante con una flecha de entrada y dos o más flechas de salida. La flecha de la transición de entrada se etiqueta con el evento disparador (si lo hay). Cada salida se etiqueta con una condición de guarda (Figura 13.119).

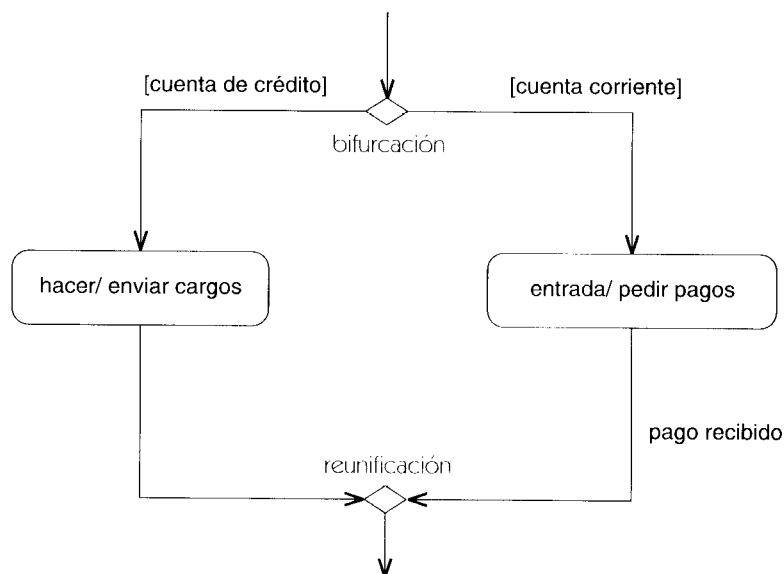


Figura 13.119 Bifurcación y reunificación

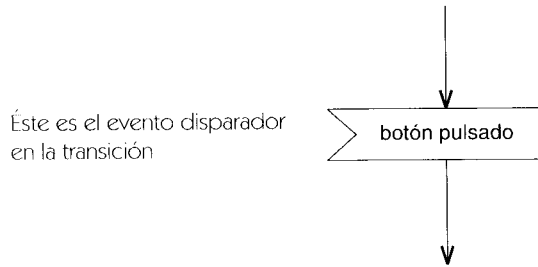


Figura 13.120 Recepción de señal

Reunificación. Una reunificación es un lugar en el cual se unen dos o más flujos de control alternativos. Es lo contrario de una bifurcación. El símbolo de una bifurcación o de una reunificación es un diamante. Es una reunificación si hay flechas múltiples de entrada; es una bifurcación si hay flechas múltiples de salida (Figura 13.119). Las reunificaciones no son estrictamente necesarias (las transiciones múltiples que pasan a un solo estado son una reunificación), sino que pueden ser visualmente útiles para mostrar cómo encajan las bifurcaciones anteriores.

Recepción de señal. La recepción de una señal se puede representar como un pentágono cóncavo que parezca un rectángulo con una muesca triangular en su lado (cualquier lado). La signatura de la señal se muestra dentro del símbolo. Se dibuja una flecha de transición sin etiqueta desde el estado anterior de la acción al pentágono, y otra flecha de transición sin etiqueta desde el pentágono al siguiente estado de acción. Este símbolo sustituye a la etiqueta del evento en la transición, que se dispara cuando la actividad anterior ha sido completada y ocurre el evento (Figura 13.120). Opcionalmente, se puede dibujar una flecha de líneas discontinuas desde un símbolo de objeto a la muesca en el pentágono para mostrar el remitente de la señal.

Envío de señal. El envío de una señal se puede representar como un pentágono convexo que parezca un rectángulo con un triángulo en un lado (cualquier lado). La signatura de la señal se muestra dentro del símbolo. Se dibuja una flecha de transición sin etiqueta desde el estado anterior de la acción al pentágono, y de otra flecha de transición sin etiqueta desde el pentágono al siguiente estado de acción.

Este símbolo sustituye la etiqueta de enviar señal en la transición (Figura 13.121). Opcionalmente, se puede dibujar una flecha con líneas discontinuas desde el extremo del pentágono a un símbolo de objeto para mostrar el receptor de la señal.

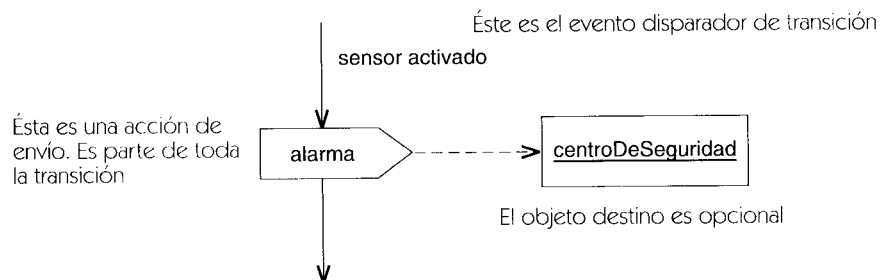


Figura 13.121 Envío de señal

Ejemplo

En la Figura 13.122, **Introducir Datos Tarjeta Crédito** y **Cargar Tarjeta** son actividades. Cuando se terminan, el proceso se mueve al paso siguiente. Después de que **Introducir Datos Tarjeta Crédito** finalice, hay una bifurcación en la cantidad de la petición; si es mayor de 25\$, se debe obtener una autorización. Se envía al centro de crédito una señal de **petición**. En una máquina de estados, esto sería representado como una acción unida a la transición que sale de **Introducir Datos Tarjeta Crédito**; ambas significan la misma cosa. Sin embargo **Esperar Autorización** es en realidad un estado de espera. No es una actividad que termina internamente. En su lugar, él debe esperar una señal externa del centro de crédito (autorización).

Cuando ocurre la señal, se dispara una transición normal y el sistema pasa a la actividad **Cargar Tarjeta**. El evento disparador se habría podido mostrar como una etiqueta en la transición de **Esperar Autorización** a **Cargar Tarjeta**. Es simplemente una variante de la notación que significa la misma cosa.

La Figura 13.123 muestra el mismo ejemplo, sin los símbolos de control especiales.

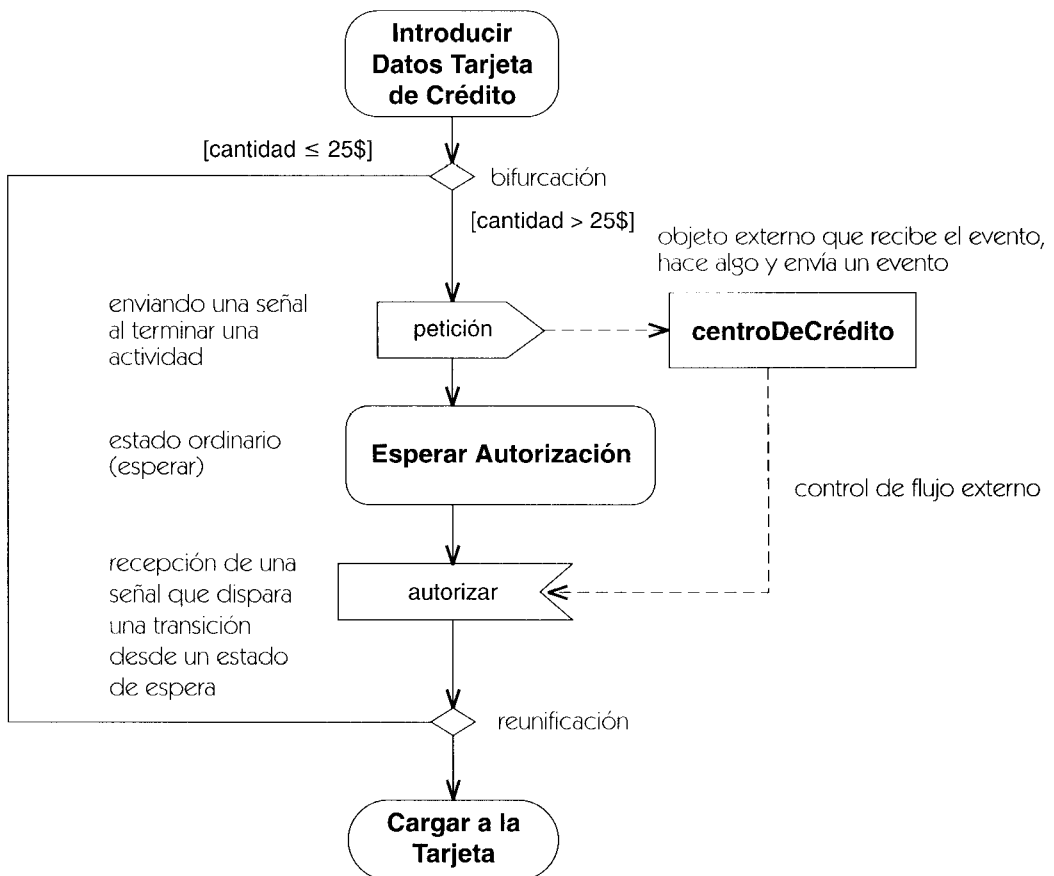


Figura 13.122 Diagrama de actividad mostrando el envío y recepción de señales

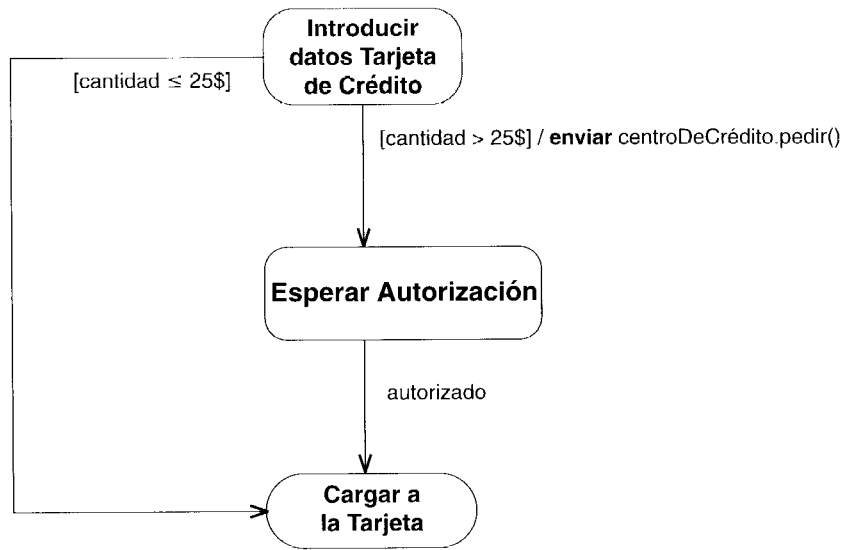


Figura 13.123 Diagrama de actividad sin símbolos especiales

identidad

Propiedad inherente de un objeto que permite distinguirlo de todos los demás objetos.

Véase también valor dato, objeto.

Semántica

Los objetos son discretos y se pueden distinguir entre sí. La identidad de un objeto es su clave conceptual, la característica inherente que permite identificar ese objeto y que otros objetos hagan referencia a él. Conceptualmente, un objeto no precisa una clave ni mecanismo adicional alguno para identificarse a sí mismo; estos mecanismos no deberían estar incluidos en los modelos. En una implementación la identidad se puede implementar mediante direcciones o claves pero éstas forman parte de la infraestructura de implementación subyacente y no es preciso incluirlos explícitamente como atributos en casi todos los modelos.

implementación

1. Una definición de la forma en que se construye o calcula algo. Por ejemplo, una clase es una implementación de un tipo; un método es una implementación de una operación. Compárese con especificación. La relación de realización es la que relaciona a una implementación con su especificación.

Véase realización.

2. Aquella fase de un sistema que describe el funcionamiento del sistema en un medio ejecutable (tal como un lenguaje de programación, una base de datos o algún hardware digital). Para la implementación es preciso hacer que las decisiones tácticas de bajo nivel adapten el diseño al medio concreto de implementación y además hay que soslayar sus limitaciones (todos los lenguajes tendrán limitaciones arbitrarias). Sin embargo, si el diseño está bien hecho entonces las decisiones de implementación serán locales y ninguna afectará a grandes segmentos del sistema. Esta fase se captura mediante modelos del nivel de implementación y especialmente en las vistas estática y de código. Compárese con análisis, diseño, implementación y despliegue.

Véase proceso de desarrollo, fases de modelado.

importar

Denota un estereotipo de la dependencia de permiso en la cual los nombres del paquete proveedor se añaden al espacio de nombres del paquete cliente.

Véase también acceso, paquete, visibilidad.

Semántica

Los nombres que se hallan en el espacio de nombres del paquete proveedor se añaden al espacio de nombres del paquete cliente, con las mismas reglas de visibilidad que se hayan especificado en el acceso. Si hay algún conflicto entre los nombres importados y los nombres que ya estén en el espacio de nombres del cliente, el modelo está mal formado.

Véase acceso para las reglas de visibilidad, tanto para el acceso como para la importación.

Notación

La dependencia de importación se representa mediante una flecha discontinua que va desde el paquete que obtiene acceso hasta el paquete que proporciona los elementos; la flecha lleva asociada la palabra clave de estereotipo «**import**».

inactivo

Un estado que no está activo; estado que no está almacenado en un objeto.

incluir

Es una relación entre un caso de uso *base* y un caso de uso *incluido*, que especifica la forma en que se puede insertar el comportamiento definido para el caso de uso incluido en el comportamiento definido para el caso de uso base. El caso de uso base puede ver la inclusión y puede

basarse en los efectos de realizar esa inclusión pero ni la base ni la inclusión pueden acceder a los atributos del otro.

Véase también extender, caso de uso, generalización de casos de uso.

Semántica

La relación de inclusión conecta un caso de uso base con un caso de uso incluido. El caso de uso incluido que figura en esta relación no es un clasificador instanciable independientemente. Lo que hace es describir explícitamente una secuencia adicional de comportamiento que se inserta en una instancia de caso de uso que está ejecutando el caso de uso base. A este mismo caso de uso base se le pueden aplicar múltiples relaciones de inclusión. El mismo caso de uso incluido se puede incluir en múltiples casos de uso base. Esto no indica ninguna relación entre los casos de uso base. Puede haber, incluso, múltiples relaciones de inclusión entre el mismo caso de uso incluido y casos de uso base, siempre y cuando cada inserción se haga en una posición diferente de la base.

El caso de uso incluido puede acceder a atributos o a operaciones del caso de uso base. La inclusión representa un comportamiento encapsulado que, potencialmente, puede reutilizarse en múltiples casos de uso base. El caso de uso base ve al caso de uso incluido, que puede dar valores a los atributos del caso de uso base. Pero el caso de uso base no debe acceder a los atributos del caso de uso incluido, porque el caso de uso incluido habrá concluido cuando el caso de uso base recupere el control.

Obsérvese que se pueden anidar las adiciones (sea cual fuere su clase). Por tanto, una inclusión puede servir como base para otra inclusión, extensión o generalización posterior.

Estructura

La relación de inclusión posee la siguiente propiedad.

localización	Es una situación, dentro del cuerpo de la secuencia de comportamiento del caso de uso base, en que se debe insertar la inclusión. Cuando una instancia de un caso de uso llega a la localización mientras está ejecutando el caso de uso base, se ejecuta el caso de uso incluido antes de reanudar el caso de uso base. La inclusión es una sentencia explícita situada dentro de la secuencia de comportamiento del caso de uso base. Por tanto la localización es implícita, a diferencia de la situación habida con la relación de extender.
--------------	--

La inclusión se efectúa una sola vez. Se pueden lograr otras multiplicidades mediante bucles en la secuencia de comportamiento correspondiente al caso base al que hace referencia la inclusión.

Notación

Se traza una flecha discontinua desde el símbolo del caso de uso base hasta el símbolo del caso de uso incluido, con una cabeza de flecha abierta en la inclusión. Se pone la palabra «**include**»

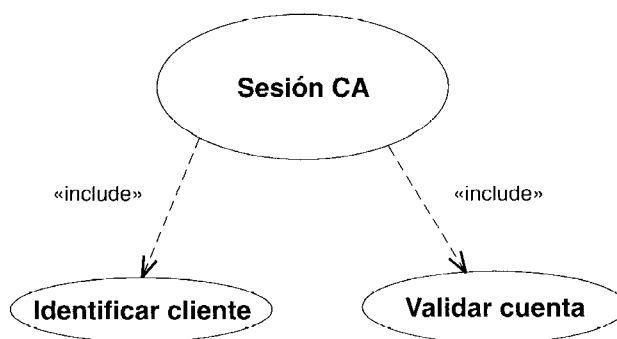


Figura 13.124 Relación de inclusión

en la flecha (Figura 13.124). Se puede asociar la localización a la flecha como una lista de propiedades entre llaves pero normalmente se hace referencia a ella formando parte del texto correspondiente al caso de uso base y no es necesario mostrarla en el diagrama. La Figura 13.125 muestra las secuencias de comportamiento para estos casos de uso.

Caso de uso base para una sesión de cajero automático:

mostrar el anuncio del día	paso de comportamiento
incluir identificación cliente	inclusión
incluir verificación de cuenta	otra inclusión
imprimir encabezado del recibo	paso de comportamiento
desconexión	paso de comportamiento

Caso de uso incluido para **Identificar cliente**:

- obtener nombre cliente
- incluir** verificar identidad
- si** fracasa la verificación **entonces** abortar la sesión
- obtener los números de cuenta del cliente

Caso de uso incluido para **Verificar cuenta**:

- establecer conexión con la base de datos de cuentas
- obtener estado de cuenta y límites

Figura 13.125 Secuencias de comportamiento para casos de uso

información de fondo

Cada aparición de un símbolo de clase en un diagrama o en diferentes diagramas puede tener distintas decisiones de presentación. Por ejemplo, el símbolo de una clase puede mostrar los atributos y operaciones y el de otra clase puede suprimirlos. Las herramientas pueden proporcionar hojas de estilo asociadas a símbolos individuales o a diagramas enteros, donde se especifiquen las decisiones de presentación, pudiéndose aplicar no sólo a las clases, sino a otros símbolos cualesquiera.

No resulta útil mostrar gráficamente toda la información de modelado. Algunas informaciones son más útiles si se representan mediante texto o en forma de tablas. Por ejemplo, la información detallada de programación a menudo está mejor presentada como listas de texto. UML no supone que toda la información de un modelo se va a expresar mediante diagramas; alguna información puede estar disponible únicamente en tablas. Este documento no intenta establecer el formato de dichas tablas ni la forma de acceder a ellas, ya que la información subyacente se describe adecuadamente en el metamodelo de UML, siendo la representación tabular responsabilidad de la herramienta. Sin embargo, se supone que pueden existir enlaces ocultos entre elementos gráficos o tabulares.

iniciación

Consiste en fijar el valor de un objeto recién creado, a saber, los valores de sus atributos, los enlaces de las asociaciones a las que pertenece y su estado de control. También se denomina inicialización.

Véase también instanciación.

Semántica

Concretamente, un objeto nuevo se crea por entero en un solo paso. Sin embargo, resulta más sencillo pensar en el proceso de instanciación como si tuviese dos pasos: creación e iniciación. En primer lugar se reserva el esqueleto vacío de un nuevo objeto, con la estructura adecuada de espacios para atributos; se le da una identidad al nuevo objeto en bruto. La identidad se puede implementar de varias maneras, tales como la dirección del bloque de memoria que contiene el objeto o bien mediante un contador entero. En todo caso, se trata de algo que resulta único en todo el sistema y se puede emplear como clave para hallar el objeto y acceder a él. En este momento, el objeto todavía no es válido: puede violar las restricciones existentes respecto a sus valores y relaciones. El paso siguiente es la iniciación. Se evalúan todas las expresiones de valores iniciales que puedan haberse declarado y se asignan los resultados a los atributos correspondientes. El método de creación puede calcular explícitamente los valores de los atributos, anulando de este modo los valores iniciales por defecto. Los valores resultantes deben satisfacer las posibles restricciones que afecten a la clase. El método de creación puede también crear enlaces que contengan al nuevo objeto. Deben satisfacer la multiplicidad declarada de todas aquellas asociaciones en que participe la clase. Una vez finalizada la iniciación, el objeto debe ser un objeto válido y tendrá que obedecer las restricciones que afecten a su clase. Una vez finalizada la iniciación, aquellos atributos o asociaciones cuya propiedad de modificabilidad sea **frozen** o **addOnly** no podrán ser alterados mientras no se destruya el objeto. Todo el proceso de iniciación es atómico y no se puede interrumpir ni entrelazar.

inicio

Es la primera fase de un proceso de desarrollo de software, durante la cual se conciben y evalúan las ideas iniciales de un sistema. En esta fase se desarrolla una parte de la vista de análisis y pequeñas porciones de otras vistas.

Véase proceso de desarrollo.

instancia

Es una entidad individual con su propio valor e identidad. Un descriptor especifica la forma y comportamiento de un conjunto de instancias cuyas propiedades son similares. Las instancias poseen identidad y unos valores que son consistentes con la especificación que consta en el descriptor. Normalmente, las instancias aparecen en los modelos como ejemplos consistentes con modelos del nivel de descriptores.

Véase también descriptor, identidad, enlace, objeto.

Semántica

Toda instancia tiene identidad. En otras palabras, dados distintos instantes de tiempo durante la ejecución de un sistema, se puede identificar una instancia con esa misma instancia en otros instantes de tiempo, aun cuando cambie el valor de la instancia. Una instancia posee, en todo momento, un valor que se puede expresar en términos de valores de datos y de referencias de otras instancias. Un valor de datos es un caso degenerado. Su identidad es lo mismo que su valor; desde un punto de vista diferente, carece de identidad.

Además de la identidad y el valor, toda instancia posee un descriptor que limita los valores que puede tener la instancia. Un descriptor es un elemento del modelo que describe instancias. Esta es la dicotomía instancia-descriptor. Casi todos los conceptos de modelado propios de UML tienen este carácter dual. El contenido principal de muchos modelos está formado por descriptores de diferentes clases. El propósito del modelo es describir los posibles valores de un sistema en términos de las instancias y de sus valores.

Cada clase de descriptor describe una clase de instancia. Un objeto es una instancia de una clase; un enlace es una instancia de una asociación. Un caso de uso describe posibles instancias de casos de uso; un parámetro describe un posible valor de argumento y así sucesivamente. Algunas instancias no poseen nombres de familia y suelen ignorarse salvo en circunstancia muy formales, pero siguen existiendo pese a todo. Por ejemplo, un estado describe posibles apariciones del estado durante el seguimiento de una ejecución.

Un modelo describe los posibles valores de un sistema y su comportamiento al pasar de uno a otro valor durante la ejecución. El valor del sistema es el conjunto de todas las instancias que contenga y de sus valores. El valor del sistema sólo es válido si toda instancia es la instancia de algún descriptor que haya en el modelo, siempre y cuando el conjunto de instancias satisfaga todas las restricciones explícitas e implícitas que haya en el modelo.

Los elementos de comportamiento del modelo describen la forma en que progresan de un valor a otro el sistema y las instancias que contiene. El concepto de identidad de instancias es esencial para esta descripción. Todo paso de comportamiento es la descripción del cambio de valores habido en un pequeño número de instancias en términos de sus valores anteriores. El resto de las instancias del sistema mantiene sus valores sin cambios. Por ejemplo, una operación local aplicada un objeto se puede describir mediante expresiones para los nuevos valores de cada uno de los atributos del objeto, sin que haya cambios en el resto del sistema. Una función no local puede descomponerse en funciones locales de varios objetos.

Obsérvese que las instancias de un sistema en ejecución no son elementos del modelo. Normalmente, no forman parte del modelo en modo alguno. Cuando aparecen instancias en un modelo, lo hacen como ilustraciones o ejemplos de la estructura y comportamiento típicos del mismo, son instantáneas del valor del sistema o el resultado de seguir la historia de su ejecución. Resultan útiles para la comprensión humana, pero suelen ser puntos de un conjunto grande o infinito de posibles valores y no *definen* nada.

Instancia directa. Todo objeto es la instancia directa de alguna clase y la instancia indirecta de los antecesores de esa clase. Esto sucede también para las instancias de otros elementos generalizables. Un objeto es una instancia directa de una clase si la clase describe la instancia y no hay ninguna clase descendiente que también describa el objeto. En el caso de la clasificación múltiple, una instancia puede ser una instancia directa de más de un clasificador, ninguno de los cuales es un antecesor de alguno de los demás. Para una cierta semántica de ejecución, se designa a uno de los clasificadores como clase de implementación y los otros se toma como tipos o roles. El descriptor completo es la descripción completa implícita de una instancia —todos sus atributos, operaciones, asociaciones y demás propiedades— tanto si los obtiene la instancia de su clasificador director como si provienen por herencia de un clasificador que sea su antecesor. En el caso de clasificación múltiple, el descriptor completo es la unión de todas las propiedades definidas por cada uno de los clasificadores directos.

Creación. Véase instanciación para una descripción de la forma en que se crean las instancias.

Notación

Aun cuando los descriptors y las instancias no son una misma cosa, comparten muchas propiedades, incluyendo una misma forma (porque el descriptor debe describir la forma de las instancias). Por tanto, resulte conveniente seleccionar una notación para cada pareja descriptor-instancia tal que su correspondencia sea visualmente obvia inmediatamente. Existe un número limitado de formas de hacer esto, cada una de las cuales posee sus propias ventajas y desventajas. En UML la distinción descriptor-instancia se muestra empleando el mismo símbolo geométrico para cada pareja de elementos y subrayando la cadena que da nombre a un elemento de instancia. Esta distinción visual es en general fácil de discernir sin resultar excesiva cuando se tiene todo un diagrama que contiene elementos de instancia.

Aun cuando la Figura 13.126 muestra objetos, la convención de subrayado puede emplearse para otras clases de instancias, tales como los casos de uso, las instancias de componentes y las instancias de nodos.

Como las instancias aparecen como ejemplos en los modelos, sólo suelen incluirse los detalles relevantes para algún ejemplo particular. Por ejemplo, no es necesario incluir toda la lista completa de atributos; además se puede omitir la lista completa de valores si la atención se centra en otra cosa, tal como el flujo de mensajes entre objetos.

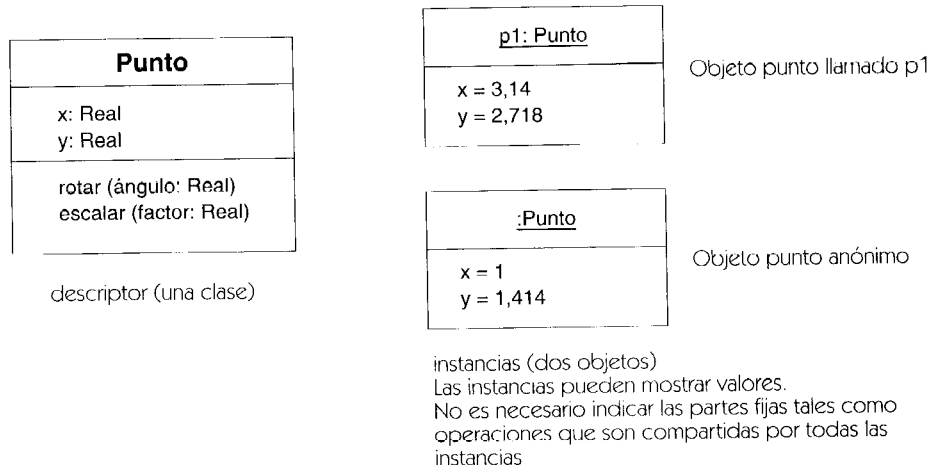


Figura 13.126 Descriptor e instancias

instanciable

Que puede poseer instancias. Es sinónimo de concreto.

Véase también abstracto, instancia directa, elemento generalizable.

Semántica

Los elementos generalizables se pueden crear como abstractos o como instanciables. Si son instanciables, entonces será posible crear instancias directas.

instanciación

Creación de nuevas instancias de elementos de un modelo.

Véase también iniciación.

Semántica

Las instancias se crean durante la ejecución como resultado de acciones primitivas de creación o de operaciones de creación. Primero se crea una identidad para la nueva instancia; después se reserva su estructura de datos según prescriba su descriptor; por último se dan valores iniciales a los valores de sus propiedades según prescriban el descriptor y el operador de creación.

La dependencia de utilización de la instanciación muestra la relación existente entre una operación que crea instancias o una clase que contiene esa operación y la clase de los objetos que se instancian.

Objetos. Cuando se instancia (se crea) un nuevo objeto, ese objeto se crea con una cierta identidad y una memoria, y luego se le dan valores iniciales. La iniciación de un objeto define los valores de sus atributos, sus asociaciones y su estado de control.

Normalmente, toda clase concreta posee una o más operaciones de constructor con alcance de clase cuyo propósito es crear nuevos objetos de la clase. Subyacente a todas las operaciones de constructor existe una operación primitiva implícita que crea una nueva instancia en blanco que después será iniciada por las operaciones de constructor. Una vez creada una instancia en blanco, ésta posee la forma prescrita por su descriptor, pero sus valores aún no han recibido valores iniciales, así que puede ser semánticamente inconsistente. Por tanto, una instancia no está disponible para el resto del sistema mientras no haya sido iniciada, lo cual sucederá inmediatamente después de la creación de la instancia en blanco.

Enlaces. Análogamente, los enlaces se crean mediante acciones u operaciones de creación, normalmente ejecutadas por operaciones de alcance de instancia asociadas a una de las clases participantes, en lugar de hacerlo mediante operaciones de constructor aplicadas al elemento de asociación en sí (aun cuando ésta es una posible técnica de implementación en algunas circunstancias). Una vez más, existe una operación primitiva implícita que crea un nuevo enlace entre una tupla específica de objetos. Esta operación no produce efectos si ya existe un enlace de la misma asociación entre la tupla de objetos (porque la extensión de una asociación es un conjunto y no puede contener valores duplicados). En una asociación ordinaria, ya no queda nada más por hacer. Un enlace de una clase asociación, sin embargo, requiere la inicialización de los valores de sus atributos.

Instancias de casos de uso. La instanciación de un caso de uso significa que se crea una instancia de caso de uso, y la instancia de caso de uso empieza a ejecutarse al principio del caso de uso que la controla. La instancia de caso de uso puede seguir temporalmente a otro caso de uso con el que esté relacionada por relaciones de extensión o inclusión antes de seguir ejecutando el caso de uso original. Cuando la instancia de caso de uso llega al final del caso de uso que está siguiendo, la instancia de caso de uso concluye.

Otras instancias. Las instancias de otros descriptores se pueden crear en un proceso análogo de dos pasos: primero se efectúa una creación en blanco para establecer la identidad y reservar la estructura de datos; luego se inician los valores de la nueva instancia para que cumpla todas las restricciones pertinentes. Por ejemplo, se crea implícitamente una activación como consecuencia directa de una llamada a una operación.

Los mecanismos exactos para la creación de instancias serán responsabilidad del entorno de ejecución.

Notación

La dependencia de instanciación se representa mediante una flecha discontinua que va desde la operación o clase que realiza la instanciación hasta la clase que se instancia; el estereotipo «**instantiate**» se asocia a la flecha.

Discusión

En algunas ocasiones se emplea el término instanciación para denotar el enlazado de una plantilla para producir un elemento enlazado, pero para esta relación es más específico el término enlazado.

instancia de

Es la relación entre una instancia y su descriptor.

Véase instancia.

instancia de caso de uso

Ejecución de una secuencia de acciones especificada en un caso de uso.

Véase caso de uso.

instancia directa

Una instancia, tal como un objeto, cuyo descriptor más específico, tal como una clase, es una clase dada.

Utilizado en una frase como, "El Objeto O es una instancia directa de la clase C." en este caso, la clase C es la clase directa del objeto O.

Véase clase directa.

instancia indirecta

Se trata de una entidad que es una instancia de un elemento, tal como una clase; además, es una instancia de un descendiente del elemento. Esto es, se trata de una instancia pero no es una instancia directa.

instanciar

Crear una instancia de un descriptor.

Véase instanciación.

instantánea

Colección de objetos, enlaces y valores que forma la configuración de un sistema en un determinado instante de su ejecución.

intención

Dícese de la especificación formal de las propiedades estructurales y de comportamiento de un descriptor. Compárese con: extensión.

Véase también descriptor.

Semántica

Un descriptor, tal como una clase o una asociación, posee tanto una descripción (su intención) como un conjunto de instancias que describe (su extensión). El propósito de la intención es especificar las propiedades estructurales y de comportamiento de las instancias en una forma ejecutable.

interacción

Se trata de la especificación de la forma en que se envían mensajes entre objetos u otras instancias para ejecutar una tarea. La interacción se define en el contexto de una colaboración.

Véase también diagrama de interacción.

Semántica

Los objetos u otras instancias de una colaboración se comunican para lograr un propósito (tal como llevar a cabo una operación) mediante el intercambio de mensajes. Entre los mensajes pueden contarse las señales y llamadas, así como interacciones más implícitas a través de condiciones o de eventos temporales. Un patrón de intercambios de mensajes que se realizan para lograr un propósito específico es lo que se denomina una interacción.

Estructura

Una interacción es una especificación de comportamiento que abarca una secuencia de intercambios de mensajes entre un conjunto de objetos, efectuada para lograr un cierto propósito tal como la implementación de una operación. Una interacción es una colaboración más un conjunto secuenciado de flujos de mensajes que se imponen a los enlaces de la colaboración. Para especificar una interacción, primero es necesario especificar una colaboración —esto es, definir los objetos que interaccionan y sus relaciones entre sí—. Entonces se especifican las posibles secuencias de interacción, bien en una única descripción que contiene condiciones (bifurcaciones o señales condicionales) o bien como múltiples descripciones, cada una de las cuales describe una de entre las posibles rutas de ejecución. La descripción completa del comportamiento de una colaboración se puede dar como una máquina de estados, cuyos estados son los estados de ejecución de una operación o de algún otro procedimiento.

Notación

Las interacciones se muestran como diagramas de secuencia o como diagramas de colaboración. Ambos formatos de diagrama muestran la ejecución de las colaboraciones. Los diagramas de secuencia son la visualización explícita del comportamiento de las colaboraciones, incluyendo la secuenciación temporal de mensajes y una representación explícita de las activaciones de métodos. Sin embargo, los diagramas de secuencias muestran únicamente los objetos participantes y no sus relaciones con otros objetos o sus atributos. Por tanto, no muestran en su to-

talidad el punto de vista contextual de una colaboración. Los diagramas de colaboración muestran todo el contexto de una interacción, incluyendo los objetos y sus relaciones relevantes para una interacción, así que suelen ser mejores que los diagramas de secuencia a efectos de diseño.

interfaz

Un conjunto de operaciones que posee un nombre y que caracteriza el comportamiento de un elemento.

Véase también clasificador, realización.

Semántica

Una interfaz es un descriptor de las operaciones visibles externamente de una clase u otra entidad (incluyendo las unidades de compendio, tales como los paquetes) que no especifica la estructura interna. Cada interfaz suele especificar únicamente una parte limitada del comportamiento de una clase real. Una clase puede admitir muchas interfaces, cuyos efectos podrán ser disjuntos o estar solapados. Las interfaces no poseen implementación; carecen de atributos, estados y asociaciones; sólo poseen operaciones y señales que reciben. Las interfaces pueden tener relaciones de generalización. Una interfaz descendiente incluye todas las operaciones y señales de sus antecesores pero puede añadir operaciones adicionales. En esencia, una interfaz equivale a una clase abstracta sin atributos ni métodos, que poseyera únicamente operaciones abstractas. Todas las operaciones de una interfaz tienen visibilidad pública (en caso contrario, no tendría sentido incluirlas, porque una interfaz no tiene nada “interno” que pueda hacer uso de ellas).

La siguiente definición extendida indica el propósito de una interfaz.

- Una interfaz es una colección de operaciones que se emplea para especificar un servicio de una clase o de un componente.
- Una interfaz sirve para nombrar una colección de operaciones y para especificar sus firmas y efectos. Una interfaz se centra en los efectos, no en la estructura, de un servicio dado. Una interfaz no ofrece una implementación para ninguna de sus operaciones. La lista de operaciones puede incluir también las señales que esa clase está dispuesta a manejar.
- Una interfaz se emplea para especificar un servicio que proporciona un proveedor y que pueden solicitar otros elementos. La interfaz da nombre a una colección de operaciones que funcionan en cooperación para realizar algún comportamiento de interés lógico de un sistema o como parte de un sistema.
- Una interfaz define un servicio que ofrece una clase o componente. Define un servicio que a su vez es implementado por una clase o componente. Como tal, una interfaz abarca los límites lógicos y físicos de un sistema. Una o más clases (que formarán probablemente parte de un subsistema componente) pueden proporcionar una implementación lógica de la interfaz. Uno o más componentes pueden proporcionar un empaquetamiento físico que se adapte a esa misma interfaz.

- Si una clase realiza (implementa) una interfaz, entonces debe declarar o heredar todas las operaciones de la interfaz. Si una clase realiza más de una interfaz, tiene que contener todas y cada una de las operaciones que se encuentren en cualquiera de sus interfaces. Una misma operación puede aparecer en más de una interfaz. Si coinciden sus firmas, deben representar la misma operación o bien estarán en conflicto y el modelo estará mal formado. (Una implementación puede adoptar reglas específicas de un lenguaje para las firmas coincidentes. Por ejemplo, en C++ los nombres de los parámetros y los tipos proporcionados se ignoran.) Una interfaz no hace afirmación alguna acerca de los atributos o asociaciones de una clase; éstos forman parte de su implementación.

Una interfaz es un elemento generalizable. Una interfaz descendiente hereda todas las operaciones de su predecesor y puede añadir algunas operaciones. La realización puede considerarse como una herencia de comportamiento; una clase hereda las operaciones de otro clasificador, pero no su estructura. Una clase puede realizar a otra clase. La clase que haga las veces de especificación actuará como interfaz en tanto en cuanto sólo sus operaciones afectan a la relación.

Las interfaces no participan en las asociaciones. Una interfaz no puede tener una asociación que sea navegable partiendo de la interfaz. Una interfaz, sin embargo, puede ser el destino de una asociación, siempre y cuando la asociación sólo sea navegable hacia la interfaz.

Notación

Una interfaz es un clasificador y se puede mostrar empleando el símbolo del rectángulo con la palabra reservada «**interface**». En el compartimento de operación se pone una lista de las operaciones que admite esa interfaz. Las señales que llevan el estereotipo «**signal**» se pueden incluir también en la lista de operaciones o bien se pueden enumerar en su propio compartimento. Se puede omitir el compartimento de atributos porque siempre está vacío.

También se puede visualizar una interfaz mediante un círculo con el nombre de la interfaz situado por debajo del símbolo. El círculo puede estar conectado mediante una línea continua con clases (u otros elementos) que lo admitan. También puede conectarse con contenedores de nivel superior, tales como los paquetes, que contengan las clases. Esto indica que la clase proporciona todas las operaciones del tipo de interfaz (y posiblemente más). La notación de círculo no muestra la lista de operaciones que admite la interfaz. Utilice el símbolo de rectángulo completo para mostrar la lista de operaciones. Una clase que utilice o requiera operaciones proporcionadas por la interfaz se puede conectar al círculo mediante una flecha discontinua que apunte al círculo. La flecha discontinua implica que la clase requiere las operaciones especificadas en la interfaz para algún propósito, pero no se exige que la clase cliente haga uso de *todas* las operaciones de la interfaz. Un servicio suele especificarse mediante una prueba de suficiencia. Si un proveedor proporciona las operaciones contenidas en un cierto conjunto de interfaces, entonces satisface los requisitos de los clientes.

La relación de realización se muestra mediante una línea discontinua con una cabeza de flecha triangular sólida (un “símbolo discontinuo de generalización”) que va desde una clase a una interfaz que admita ésta. Se trata de la misma notación que se emplea para indicar la realización de un tipo por parte de una clase de implementación. De hecho, este símbolo se puede emplear entre dos símbolos de clasificador, indicando que el cliente (situado en la cola de la flecha) admite todas las operaciones definidas en el proveedor (situado en la cabeza de la

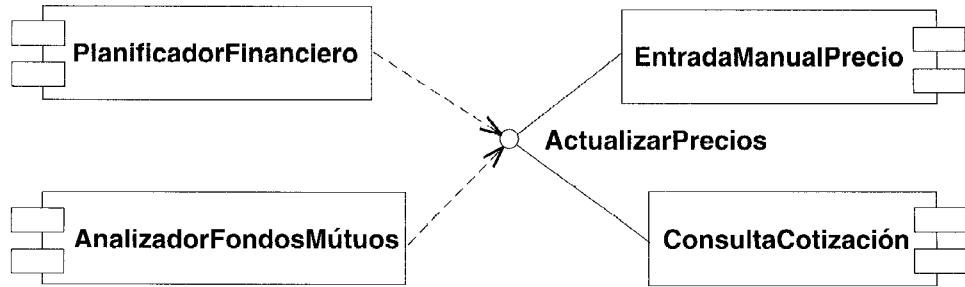


Figura 13.127 Proveedores y clientes de la interfaz

flecha), pero sin necesidad de admitir ninguna estructura de datos del proveedor (atributos y asociaciones).

Ejemplo

La Figura 13.127 muestra una vista simplificada de los componentes financieros que intervienen en los precios de las acciones de bolsa. El **PlanificadorFinanciero** es una aplicación financiera personal que administra nuestras inversiones, así como los gastos personales. El **AnalizadorFondosPensiones** examina detalladamente los fondos de pensiones. Necesita poseer la capacidad de actualizar los precios de los valores subyacentes, así como lo precios de los fondos. La capacidad de actualizar los precios de los valores se muestra mediante la interfaz **ActualizarPrecios**. Hay dos componentes que implementan esta interfaz; se muestran mediante las líneas continuas que los unen al símbolo de interfaz. El componente **EntradaManualPrecio** permite al usuario introducir manualmente los precios de los valores seleccionados. El componente **ConsultaCotización** recupera los precios de esos valores en un servidor bursátil, empleando un módem o Internet.

La Figura 13.128 muestra la notación completa de una interfaz como palabra reservada en un símbolo de clase. Se observará que esta interfaz implica dos operaciones —preguntar el precio de unas acciones y conseguir una cotización—; y además proporcionar una lista de valores de bolsa y recibir una lista de los precios que hayan cambiado. En este diagrama el componente **ConsultaCotización** está conectado a la interfaz empleando una flecha de realización, pero se trata de la misma relación que se mostraba en el diagrama anterior; sólo se está empleando una notación más explícita.

Este diagrama muestra también una nueva interfaz, **Actualizar Periódica Precios**, que es un descendiente de la interfaz original. Hereda las dos operaciones y agrega una tercera operación que envía una solicitud de actualización periódica y automática de los precios. Esta interfaz es realizada por el componente **ServidorCotización**, un servicio de suscripción. Implementa las dos mismas operaciones que **ConsultaCotización** pero de una manera diferente. No comparte la implementación de **ConsultaCotización** (en este ejemplo) y por tanto no hereda de él una implementación.

La Figura 13.128 muestra la diferencia entre herencia de interfaz y herencia completa. Esa última implica a la primera, pero una interfaz descendiente puede ser implementada de una forma distinta de la interfaz predecesora. **ServidorCotización** admite la interfaz que implementa

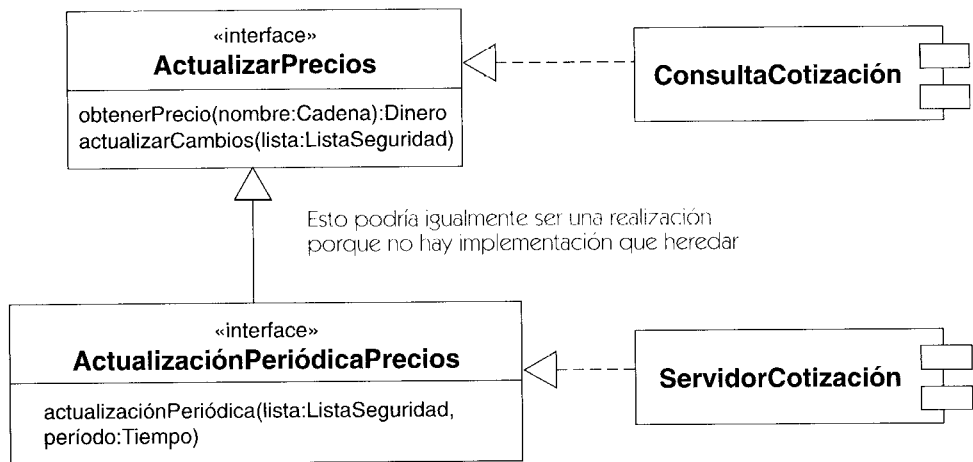


Figura 13.128 La notación de interfaz completa

ConsultaCotización, a saber, **ActualizarPrecios**, pero no hereda la implementación de **ConsultaCotización**. (En general, resulta cómodo heredar la implementación además de la interfaz, así que las dos jerarquías suelen ser idénticas.)

Una interfaz también puede contener una lista de las señales que maneja (Figura 13.129).

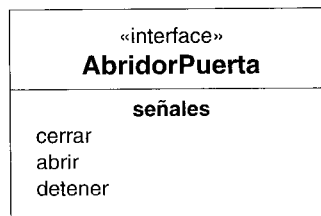


Figura 13.129 Una interfaz con señales

Las interfaces se emplean para definir el comportamiento de las clases, así como el de los componentes, sin restringir la implementación. Esto permite distinguir la herencia de interfaz, tal como se declara en Java, de la herencia de implementación.

invariante

Una restricción que forzosamente ha de ser cierta en todo momento (o por lo menos en todo momento en que no haya ninguna operación sin finalizar).

Semántica

El invariante es una expresión booleana que debe ser cierta siempre que no haya ninguna operación activa. Se trata de una aserción y no de una sentencia ejecutable. Dependiendo de la

forma exacta de la expresión, puede ser o no posible verificarla automáticamente por anticipado.

Véase también precondition, postcondición.

Estructura

Los invariantes se modelan como una restricción con el estereotipo «**invariant**» asociado al elemento.

Notación

Se puede mostrar una postcondición en una nota con la palabra reservada «**invariant**». La nota está asociada a un clasificador, atributo u otro elemento.

ligadura

Asignación de valores a parámetros para producir un elemento individual a partir de un elemento parametrizado. La relación de ligadura es un tipo de dependencia, que se utiliza para ligar plantillas y así producir nuevos elementos del modelo.

Véase también elemento ligado, plantilla.

Semántica

Una definición parametrizada, como una operación, señal o plantilla, define la forma de un elemento. Un elemento parametrizado no se puede utilizar directamente, ya que sus parámetros no tienen valores específicos. La ligadura es una dependencia que representa la asignación de valores a parámetros para producir un nuevo elemento que pueda ser utilizado. La ligadura actúa sobre las operaciones para producir llamadas, sobre las señales para producir señales de envío y sobre las plantillas para producir nuevos elementos del modelo. En las dos primeras, la ligadura se produce durante la ejecución para producir entidades en tiempo de ejecución que normalmente no figuran en los modelos excepto como ejemplos o resultados de la ejecución. Los valores de los argumentos se definen dentro del sistema de ejecución.

Sin embargo, una plantilla se liga en tiempo de modelado para producir nuevos elementos utilizables en el modelo. Los valores de los argumentos pueden ser otros elementos, como clases, además de valores dato, como cadenas o enteros. La relación de ligadura liga valores a una plantilla produciendo un elemento real del modelo que puede utilizarse directamente dentro del mismo.

Una relación de ligadura tiene un elemento proveedor (la plantilla), un cliente (el elemento recién generado), y una lista de valores para ligarlos con los parámetros de la plantilla. El

elemento ligado se define sustituyendo cada parámetro por el valor del correspondiente argumento en una copia del cuerpo de la plantilla. La clasificación de cada argumento debe ser la misma o un descendiente de la clasificación declarada de su parámetro.

Las ligaduras no afectan a la plantilla, pues se puede ligar cada plantilla muchas veces para producir cada vez un nuevo elemento ligado.

Notación

La ligadura se indica mediante la palabra clave «**bind**» asociada a una flecha discontinua que conecta el elemento generado (en el origen de la flecha) con la plantilla (en la punta de la flecha). Los valores reales de los argumentos se representan mediante una lista de expresiones de texto separadas por comas encerrada entre paréntesis a continuación de la palabra «**bind**».

Una notación alternativa y más compacta de la ligadura emplea la correspondencia de nombres para evitar la necesidad de flechas. Para representar un elemento generado mediante ligadura, se añade al nombre de la plantilla una lista de expresiones de texto separadas por comas encerrada entre los símbolos menor y mayor que (<argumento_{lista}>).

En cualquier caso, cada argumento se representa mediante una cadena de texto que se evalúa estáticamente en el momento de construir el modelo. No se evalúa dinámicamente como las operaciones o los argumentos de tipo señal.

En la Figura 13.130 se declara explícitamente (utilizando flechas) una nueva clase **ListaDirecciones**, cuyo nombre puede utilizarse en modelos y expresiones. La forma implícita **Farray<Punto,3>** declara una «clase anónima» que no tiene nombre por sí misma, y que puede utilizarse en expresiones utilizando la sintaxis en línea. En ningún caso se pueden declarar atributos u operaciones adicionales: si se necesitan extensiones debe declararse una subclase.

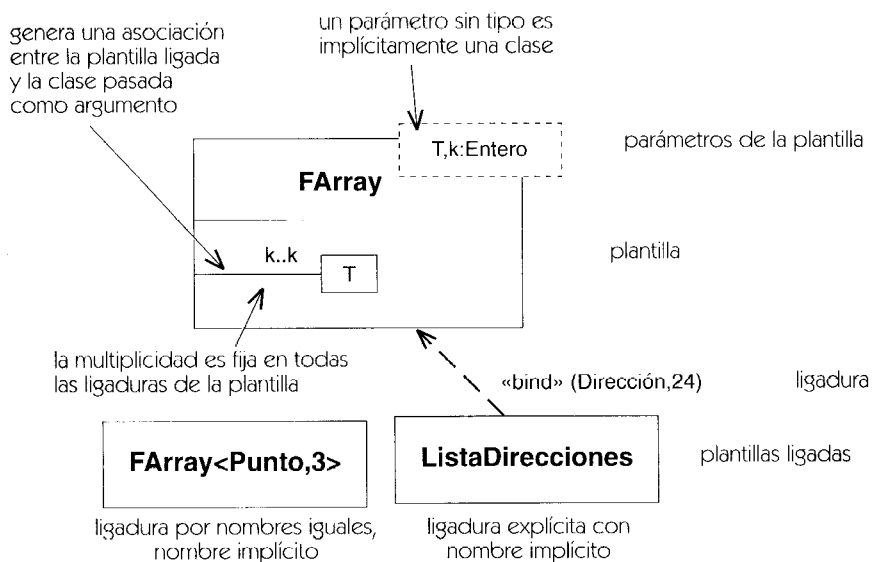


Figura 13.130 Plantillas: declaración y ligadura